BRAIN TUMOR MRI IMAGE GENERATION AND CLASSIFICATION USING A HYBRID CONDITIONAL WGAN-GP

NeuroVision AI: a research prototype web application

a micro project e-report Submitted by

Ajai Kumar N (URK23CS5064)

In partial fulfilment of the Continuous Internal Examination

of

25AI2004 – Artificial Intelligence in Healthcare

in

B. TECH COMPUTER SCIENCE AND ENGINEERING [AI]

under the supervision of

Ms. Gethsia P



KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Deemed-to-be-University)
Karunya Nagar, Coimbatore - 641 114.
October 2026

BONAFIDE CERTIFICATE

This is to certify that the micro project e-report titled **"Brain Tumor MRI Image Generation and Classification Using a Hybrid Conditional WGAN-GP"** is the bonafide work of "**Ajai Kumar N (URK23CS5064)"** who carried out the project work under my supervision.

COURSE FACULTY

Ms. Gethsia P

Assistant professor

Division of Artificial Intelligence and Machine Learning

.

Submitted for the micro project presentation scheduled on 29/10/26.

ABSTRACT

Brain tumour MRI classification needs many labelled images, but medical data are scarce, imbalanced and hard to share, and generative adversarial networks (GANs) are often proposed to create extra training images. This micro project compares three GANs on 64×64 grayscale brain MRI slices from a four-class public dataset (glioma, meningioma, pituitary tumour, no tumour): a DCGAN, an unconditional WGAN-GP and a proposed conditional WGAN-GP (cWGAN-GP) with a projection critic. An audit showed that 1,076 of the 1,600 images in the dataset's original Testing folder had a near-identical copy in the Training folder, so the 7,200 images were grouped into 3,479 near-duplicate clusters, 207 images were quarantined, and a cluster-disjoint split of 3,295 unique images into 2,311 training, 492 validation and 492 test images was built. Generation quality was measured with the Fréchet and Kernel Inception Distances (FID, KID) against the test images, and usefulness with a small CNN trained on (A) real images only, (B) real plus cWGAN-GP images and (C) a real-only control resampled to B's size and class mix. DCGAN gave the best generation scores (FID 154.7; KID×1000 129.4), ahead of cWGAN-GP (177.8; 157.6) and WGAN-GP (182.1; 163.5). Mean test accuracy was 0.817 for A, 0.839 for B and 0.869 for C: the matched real-only control scored highest, so the study shows no benefit of synthetic augmentation beyond added training volume. The work is packaged as NeuroVision AI, a React and FastAPI web application that displays the saved results, generates synthetic samples with the saved cWGAN-GP checkpoint and demonstrates a third-party pretrained classifier. It is an academic prototype, not a clinically validated system: the test set is small (492 images, 46 no-tumour), patient-level independence could not be established, and the generator did not demonstrate reliable tumour-specific anatomy.

Keywords: brain tumour MRI, generative adversarial network, conditional WGAN-GP, data leakage, near-duplicate detection, FID, KID, medical image synthesis, healthcare AI, web application

TABLE OF CONTENTS

S.No.	Title		Page No.
	BONAFIDE CERTIFICATE		ii
	ABSTRACT		iii
	TABLE OF CONTENTS		iv
1	INTRODUCTION		1
	1.1	Background of the selected problem	1
	1.2	Industry Relevance	1
	1.3	Aim and Objective of the Micro Project	2
	1.4	Link to SDG / KITS Technology Mission	3
2	LITERATURE REVIEW		4
3	METHODOLOGY		7
4	FABRICATION/SIMULATION/PROGRAM OR CODE/DESIGN		14
5	RESULTS AND INTERPRETATION		22
6	CONCLUSION AND FUTURE SCOPE		29
	REFERENCES		31
A	APPENDIX A: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)		32
	A.1	Introduction	32
	A.2	Overall Description	33
	A.3	Specific Requirements (functional, non-functional, traceability)	35
B	APPENDIX B: DESIGN DOCUMENT		39
	B.1	Design Overview	39
	B.2	UML Diagrams	40
	B.3	Algorithms	47
	B.4	User Interface Design	49

1. INTRODUCTION

1.1 Background of the selected problem

A brain tumour is an abnormal growth of cells in or around the brain. Magnetic resonance imaging (MRI) is one of the main tools used to examine the brain, and computer-aided analysis of MRI slices is an active research area. This project works with four image classes: **glioma** and **meningioma** (two common tumour types), **pituitary tumour** (a tumour of the pituitary gland) and **no tumour**. Deep learning classifiers can learn to separate such classes, but they need many correctly labelled images. In medicine, labelled images are expensive because they need expert review, they are protected by privacy rules, rare conditions give few examples, and the classes are usually imbalanced.

One proposed remedy is **synthetic data generation**: a generative model learns the appearance of the real images and produces new ones that can be added to the training set ("data augmentation"). A **generative adversarial network (GAN)** [1] trains two neural networks against each other: a *generator* turns random noise into an image and a *discriminator* tries to tell generated images from real ones. A *conditional* GAN [2] also receives a label, so that the class of the generated image can be requested. Popular variants include the deep convolutional GAN (DCGAN) [3], the Wasserstein GAN [4] and its gradient-penalty form, WGAN-GP [5]. Generated images are compared with real ones using distribution distances such as the Fréchet Inception Distance (FID) [6] and the Kernel Inception Distance (KID) [7], and class information can be injected into the critic with a projection layer [8].

Synthetic medical images must be treated with care. A generator can produce images that merely *look* like MRI without carrying real pathology, and the requested label is not a diagnosis. A second problem concerns evaluation: if the same slice (or a mirrored or re-encoded copy of it) appears in both the training and the test data, every reported score is optimistic. This project found exactly that problem in the public dataset it started from, and therefore rebuilt its experiments on a leakage-controlled split. The project is titled *Brain Tumor MRI Image Generation and Classification Using a Hybrid Conditional WGAN-GP*; the web application built around it is called **NeuroVision AI**.

1.2 Industry Relevance

Healthcare AI, medical-image computing and medical-software engineering all face the same questions that this micro project works through on a small scale:

- **Healthcare AI and data augmentation research.** Whether synthetic images help a classifier is an open question; a fair test needs a control that gives the classifier the same amount of *real* data (condition C in this project).

- **Medical-image computing.** Duplicate and near-duplicate slices are a known risk when many slices come from the same scans. A reproducible audit and a cluster-disjoint split are practical tools for any imaging team.
- **Responsible evaluation.** The project reports uncertainty (seed spread, small test set), keeps validation and test data separate, states what cannot be concluded, and distinguishes research results from application demonstrations.
- **Software delivery.** NeuroVision AI shows how saved research results and model checkpoints can be exposed through a validated API and a browser interface for exploration by non-programmers.

These are research and engineering benefits only. The models and the application are **not approved or validated for clinical use** and must not be used to diagnose or treat patients.

1.3 Aim and Objective of the Micro Project

Aim. To compare three GANs (DCGAN, WGAN-GP and the proposed conditional WGAN-GP) for generating 64×64 brain MRI slices on a leakage-controlled dataset, to test honestly whether their synthetic images improve a downstream tumour classifier, and to deliver the completed work as a documented web application.

The measurable objectives were:

O1. Audit the dataset for exact and near-duplicate images and build a cluster-disjoint train/ validation/test split (measured by the number of clusters, quarantined images, partition sizes and the number of clusters crossing partitions, which must be zero).

O2. Implement and train DCGAN, WGAN-GP and cWGAN-GP on the training partition only, with identical data, seed, batch size and 3,000 generator updates each.

O3. Evaluate generation quality with FID and KID against the 492 test images, using three generation seeds and a real-versus-real reference floor.

O4. Implement label-conditional generation and examine whether the requested class changes and matches the generated images (per-class KID).

O5. Compare a small CNN trained on real data only (A), real plus synthetic data (B) and a size-matched real-only control (C) over three training seeds, reporting accuracy, macro-F1 and, from the saved predictions, per-class sensitivity and specificity.

O6. Build and test NeuroVision AI, a React + TypeScript frontend with a FastAPI + PyTorch backend, for viewing the saved results, generating samples and demonstrating classification, with visible research-prototype warnings.

1.4 Link to SDG / KITS Technology Mission

UN Sustainable Development Goals. The project relates most directly to **SDG 3, Good Health and Well-being** ("Ensure healthy lives and promote well-being for all at all ages") [9]. Methods that make diagnostic-support research less dependent on large private image collections, and that expose data leakage, contribute to the research base of that goal; the project itself makes no claim of health impact. It relates secondarily to **SDG 9, Industry, Innovation and Infrastructure** ("Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation") [9], through a reproducible pipeline and a working software prototype. These links are about research direction, not about delivered outcomes.

Karunya Institute of Technology and Sciences. The university's official site states its mission as raising scientists and entrepreneurs to find solutions, and names food, water, health care and energy as its product-development focus areas [10]; this project falls under the health care area. The sources consulted (karunya.edu and kcode.karunya.edu) did not contain a description of a specific "Technology Mission", so this report does not describe one and does not claim any institutional endorsement. *If the course requires a particular Technology Mission statement, it should be taken from the official document and added here.*

2. LITERATURE REVIEW

This chapter reviews the original papers on which the project's methods and metrics rest. It is a review of foundational methods, not a systematic survey of medical GAN applications. For each work the text states what it contributes, how it is used here, and what remains unresolved for low-resolution grayscale MRI.

2.1 Generative Adversarial Networks (Goodfellow et al., 2014) [1]

Goodfellow and colleagues introduced a framework in which a generator and a discriminator are trained simultaneously in a two-player game: the generator tries to produce samples the discriminator accepts as real, and the discriminator tries to reject them. The approach can be trained with back-propagation, without Markov chains or approximate-inference networks. **Relevance:** it is the basis of all three models here; the DCGAN baseline uses the standard binary cross-entropy (BCE) adversarial objective. **Unresolved:** adversarial training can be unstable, and the framework offers no built-in measure of sample quality or of medical correctness.

2.2 Conditional GANs (Mirza and Osindero, 2014) [2]

Mirza and Osindero extended the GAN by giving both networks extra information, such as a class label, so that generation can be directed. **Relevance:** the proposed cWGAN-GP feeds a label embedding to the generator together with the noise vector. **Unresolved:** conditioning does not guarantee that the model learns class-specific features; with noisy labels or few examples per class it may only change global appearance. In this dataset, one cluster family carried conflicting class labels, which makes the question sharper.

2.3 DCGAN (Radford, Metz and Chintala, 2015) [3]

Radford and colleagues proposed architectural guidelines that made convolutional GANs train more reliably: strided convolutions in place of pooling, batch normalisation, no fully connected hidden layers, ReLU in the generator and LeakyReLU in the discriminator. **Relevance:** the project's generator and discriminator at 64×64 follow these guidelines. **Unresolved:** the guidelines were demonstrated on large natural-image collections; behaviour on a few thousand grayscale MRI slices is not guaranteed.

2.4 Wasserstein GAN and WGAN-GP (Arjovsky et al., 2017; Gulrajani et al., 2017) [4], [5]

Arjovsky, Chintala and Bottou replaced the original divergence by the Wasserstein distance and used a *critic* (a discriminator without a sigmoid) constrained to be Lipschitz by weight clipping [4]. Gulrajani and colleagues showed that clipping can cause problems and proposed a **gradient penalty** that pushes the norm of the critic's gradient towards one on interpolated samples; they also recommend not using batch normalisation in the critic [5]. **Relevance:** the WGAN-GP and cWGAN-GP here use a penalty weight of 10 and five critic updates per generator update.

Unresolved: the extra critic updates cost compute, the critic loss does not directly give an image-quality number, and in this project WGAN-GP did not beat the simpler DCGAN within 3,000 generator updates.

2.5 Fréchet Inception Distance (Heusel et al., 2017) [6]

Heusel and colleagues introduced the FID, which fits Gaussians to Inception-network features of real and generated images and measures the Fréchet distance between them; lower is better. **Relevance:** FID is the main generation-quality score here. **Unresolved:** the Inception network was trained on colour photographs of everyday objects, FID is biased upward for small samples, and a low FID is not evidence of anatomical correctness. This project computes it on 492 grayscale 64×64 images.

2.6 Kernel Inception Distance (Bińkowski et al., 2018) [7]

Bińkowski, Sutherland, Arbel and Gretton analysed MMD-based GANs and proposed the KID, a squared maximum-mean-discrepancy on Inception features with a polynomial kernel and an *unbiased* estimator, which suits smaller sample sizes. **Relevance:** KID complements FID and is computed over 100 subsets of 300 images. **Unresolved:** it shares FID's domain gap for medical images, and its unbiased estimate can be slightly negative (as it is for the real-versus-real reference here).

2.7 Projection discriminators (Miyato and Koyama, 2018) [8]

Miyato and Koyama proposed a conditional discriminator whose output is an unconditional score plus the inner product of a learned class embedding with the image features. **Relevance:** the project's critic uses exactly this projection form, $D(x, y) = \psi(h(x)) + E(y) \cdot h(x)$, without batch normalisation. **Unresolved:** the method was evaluated on large natural-image datasets; whether a projection critic yields class-faithful synthetic MRI from only 227 no-tumour and a few hundred other images per class is the question this project examines, with a cautious answer in Chapter 5.

2.8 Summary and research gap

Table 2.1: Summary of the reviewed work and its role in this project.

Work	Key contribution	Used here as	Open issue for MRI
GAN [1]	Adversarial generator–discriminator training	DCGAN baseline objective	Instability; no quality or clinical check
cGAN [2]	Conditioning on a label	Label embedding in the generator	Labels may not be learned as class features
DCGAN [3]	Stable convolutional architecture	64×64 generator/ discriminator	Designed for large natural-image sets
WGAN [4], WGAN-GP [5]	Wasserstein critic; gradient penalty	WGAN-GP and cWGAN-GP training	More compute; no better than DCGAN here
FID [6]	Inception-feature Fréchet distance	Main quality score	ImageNet features; small-sample bias
KID [7]	Unbiased MMD on Inception features	Second quality score	Same domain gap; can be negative
Projection critic [8]	Class embedding projected on features	cWGAN-GP critic	Needs reliable labels and enough data

Gap addressed. The reviewed methods supply the building blocks, but they do not answer three practical questions for small grayscale medical datasets: (i) whether label-conditional WGAN-GP images are class-faithful at 64×64; (ii) whether synthetic augmentation still helps once the *amount* of real training data is controlled; and (iii) whether reported gains survive removal of near-duplicate images between training and test data. The experiments in Chapters 3 and 5 are designed around these three questions.

3. METHODOLOGY

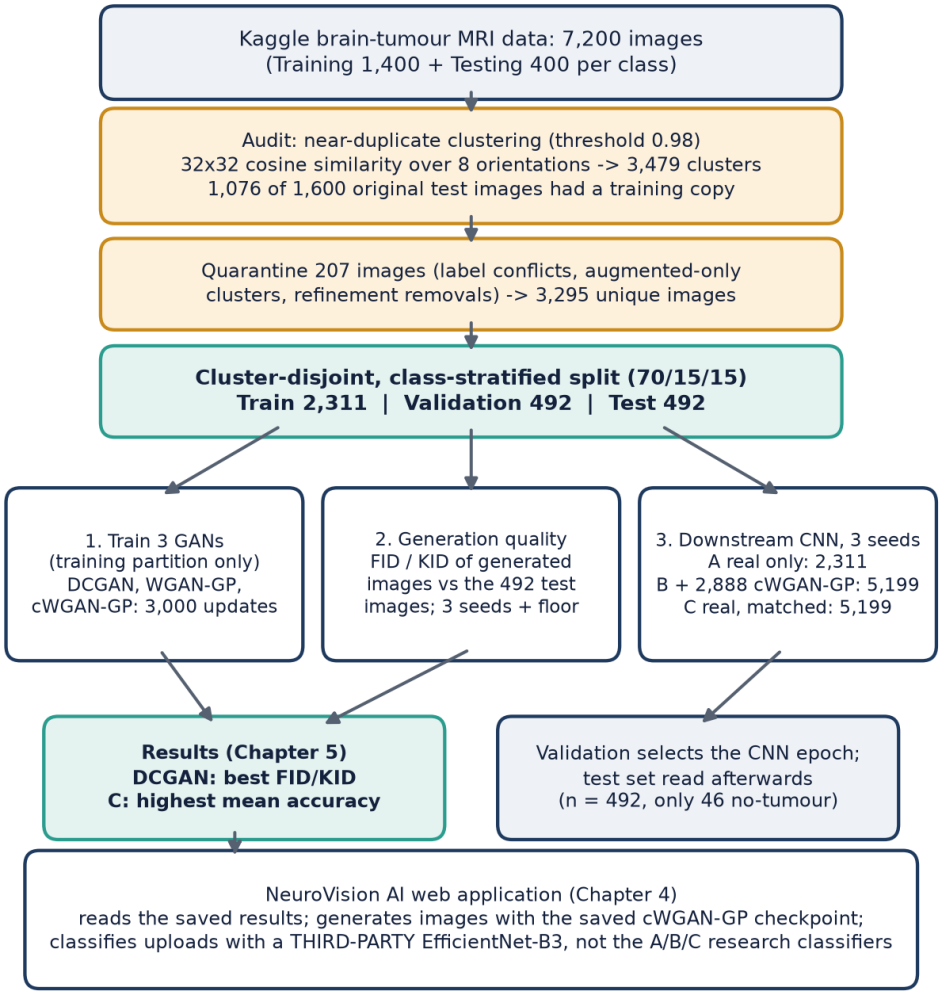


Figure 3.1: Experimental workflow of the project, from the original dataset to the results and the NeuroVision AI application.

3.1 Dataset source and initial structure

The data are a four-class brain MRI collection obtained from Kaggle. Its folder structure (Training and Testing, each with glioma, meningioma, notumor and pituitary folders), its 7,200 image files (1,400 training and 400 testing images per class) and its size (167.85 MB) match the Kaggle page *Brain Tumor MRI Dataset* by Masoud Nickparvar, Version 2 [11]. The page lists the licence as **CC BY 4.0** and names three upstream sources: the figshare brain MRI dataset, the SARTAJ dataset and the Br35H dataset (the no-tumour images come from Br35H). File names carry the prefixes Tr- and Te-, and 203 files carry an *aug* marker (100 in Training and 103 in Testing), which the Kaggle page does not mention.

Provenance caveats

The match with the Kaggle page is on checkable counts and size only. **The local download record is unverified** (no URL was saved), so it cannot be proved that the files are identical to that version.

The CC BY 4.0 licence is the one shown on the dataset page; the upstream datasets may carry separate terms. **The dataset is not redistributed** with this report or the software package; attribution: M. Nickparvar, *Brain Tumor MRI Dataset*, Kaggle.

The page states that Version 2 removed duplicates and eliminated train/test overlap. The project's own near-duplicate audit (a heuristic, described next) nevertheless found near-identical images across the two folders.

3.2 Data auditing, duplicates and image preprocessing

All 7,200 files were pooled and checked for byte-identical files (153 groups of identical files were found, which exact SHA-256 hashing detects). Exact hashing, however, cannot find *mirrored*, rotated or re-encoded copies of the same slice, which are common in this dataset. Images used by the GANs and the classifier were converted to one-channel grayscale, resized to **64×64** pixels, and scaled to the range [−1, 1] (the range of the generator's Tanh output). For FID/KID, real and generated images were converted identically (grayscale, 64×64, 8-bit, replicated to three channels) so that the Inception network sees both in the same way.

3.3 Near-duplicate detection and cluster-disjoint partitioning

Each image was reduced to 32×32 grayscale, mean-removed and length-normalised; the similarity of two images is the cosine similarity, evaluated against the second image in all eight orientations (identity, horizontal and vertical flips, three rotations and two transposes). Two images were linked when the similarity was at least 0.98 or the files were byte-identical, and linked images were merged into **clusters** (connected components). A second stage re-scored the closest held-out pairs at 96×96 pixels with shifts of ±4 pixels; pairs that looked alike at 32×32 scored about 0.94 at higher resolution, i.e. similar but distinct scans.

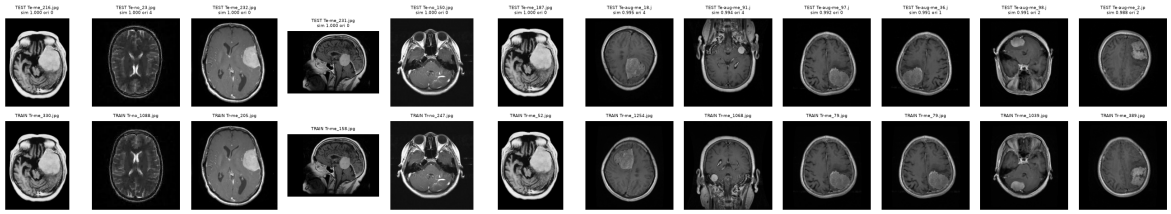


Figure 3.2: Examples from the audit of the original folders: Testing images (top row) and their near-identical, often mirrored, copies in Training (bottom row). These are real dataset images shown for audit purposes (dataset licence: CC BY 4.0 [11]).

Table 3.1: Result of the duplicate audit and cluster construction.

Quantity	Value
Candidate images pooled (Training + Testing)	7,200
Original Testing images with a near-identical Training copy	1,076 of 1,600
Near-duplicate clusters at threshold 0.98 (largest cluster: 83 images)	3,479
Clusters containing images from both original folders	756
Quarantined images: 8 clusters with conflicting class labels (30 images), 172 clusters made only of *aug* files, 4 images removed by the second stage	207
Final usable unique images (one representative per cluster)	3,295
Clusters that appear in more than one partition	0

One representative per cluster (preferring a non-augmented, originally-Training file) was assigned to train, validation or test by a class-stratified 70/15/15 split with seed 42, so that no cluster crosses a partition. The split manifest and the three file lists are hash-recorded and were re-verified by automated checks.

Table 3.2: Final cluster-disjoint partitions (unique images per class).

Partition	Glioma	Meningioma	No tumour	Pituitary	Total
Train	918	566	227	600	2,311
Val	196	122	46	128	492
Test	196	122	46	128	492

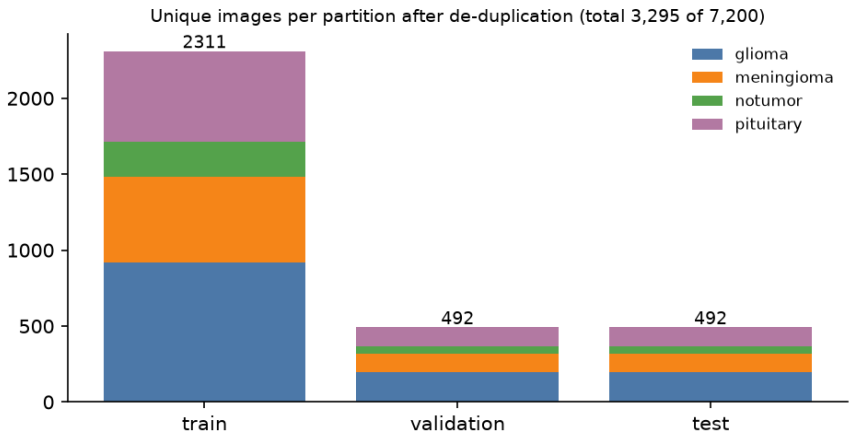


Figure 3.3: Unique images per partition and class after de-duplication.

Residual uncertainty

The similarity measure is a heuristic with no ground truth. The dataset has **no patient identifiers**, so different slices of the *same patient* cannot be detected and may lie in different partitions: patient-level independence is **not proven**. Files marked *aug* were removed by name only, and heavily rescaled or re-cropped copies below the similarity threshold would be missed. The split is therefore better protected against copy-level leakage than the original folders, not proven leak-free.

The cost of the audit is data: only 3,295 of the 7,200 images are unique and usable, and the classes became imbalanced (227 no-tumour training images, 46 in each of validation and test).

3.4 DCGAN, WGAN-GP and conditional WGAN-GP architectures

All three models generate 64×64×1 images from a 100-dimensional noise vector with the same transposed-convolution generator (channel widths 512, 256, 128, 64, 1; batch normalisation and ReLU in the hidden layers; Tanh output). They differ in the discriminator or critic, the objective and the conditioning (Table 3.3).

Table 3.3: Comparison of the three GANs.

	DCGAN [3]	**WGAN-GP [4], [5]**	**cWGAN-GP (proposed) [2], [5], [8]**
Generator	Noise z (100-d) → transposed convolutions	Same	Label embedding (50-d) concatenated to z; same convolution stack
Discriminator / critic	Convolutions with batch normalisation; outputs a logit	Convolutions without batch normalisation; scalar score	Projection critic: $D(x, y) = \psi(h(x)) + E(y) \cdot h(x)$; no batch normalisation
Objective	Non-saturating BCE	Wasserstein loss + gradient penalty ($\lambda = 10$)	Same loss; real and fake samples share the label
Optimiser [12]	Adam 2×10^{-4}, $\beta = (0.5, 0.999)$	Adam 1×10^{-4}, $\beta = (0.5, 0.9)$	Adam 1×10^{-4}, $\beta = (0.5, 0.9)$
Updates for 3,000 generator steps	3,000 discriminator	15,000 critic (5 per generator step)	15,000 critic (5 per generator step)
Conditioning	None	None	Requested class label; no classifier guidance

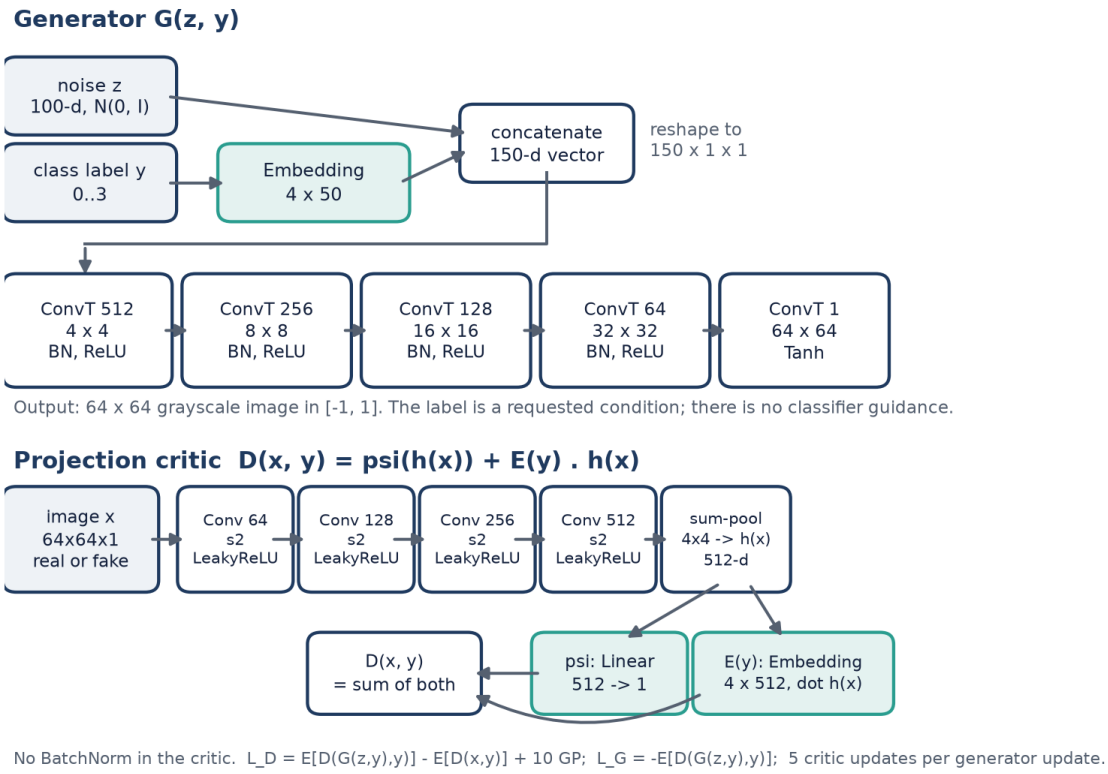


Figure 3.4: Architecture of the proposed cWGAN-GP: label-conditioned generator and projection critic.

3.5 Training configuration and hardware

Table 3.4: Training configuration and hardware.

Item	Setting
Training data	The 2,311-image training partition only (hash-verified for each model); validation and test images are never used for GAN training
Common GAN settings	64×64 grayscale, noise dimension 100, batch size 16, seed 42, 3,000 generator updates, no data augmentation
Runtime on the Apple GPU (MPS)	DCGAN 293 s; WGAN-GP 1759 s; cWGAN-GP 2149 s (cumulative; Low Power Mode was on when checked)
Hardware	Apple MacBook Air, M4 chip, 16 GB unified memory, macOS
Software	Python 3.14.3, PyTorch 2.14.0 (MPS backend, no CPU fallback), torchvision 0.29.1, torch-fidelity 0.4.0 [13]
Interrupted run	The first cWGAN-GP attempt stopped at generator step 2,861 with a non-finite loss; **the cause is unconfirmed**. It was resumed from the verified step-2,500 checkpoint and completed, so the final model is not bit-identical to an uninterrupted run. The abandoned attempt's logs are preserved.

3.6 FID/KID generation-quality evaluation

Generated images were compared with the 492 real **test** images using torch-fidelity 0.4.0 [13] with its Inception-v3 "pool3" (2,048-dimensional) features and identical preprocessing for real and generated images. Each model generated 492 images per seed over three generation seeds (1001–1003); the conditional model generated the test partition's class counts (196, 122, 46, 128). KID used 100 subsets of 300 images. A **real-versus-real floor** (class-matched random training images against the test images) shows the score of images that really are from the same distribution. The standard deviations reported later describe *generation-seed* variability only, because each GAN was trained once.

3.7 Three-condition downstream classifier experiment (A, B and C)

A small convolutional network (four convolution blocks of 32, 64, 128 and 256 filters, global average pooling, dropout 0.3, Adam with learning rate 10^{-3}, batch size 64, 15 epochs, no augmentation) was trained with three seeds (42–44). The validation partition selects the best epoch; the test partition is read only afterwards. The three conditions differ only in the training set:

Table 3.5: Training sets of the three downstream conditions.

Condition	Training images	Per-class counts (glioma, meningioma, no tumour, pituitary)	Total
A: real only	Real training images	918, 566, 227, 600	2,311
B: real + synthetic	A + 722 cWGAN-GP images per class (labels = requested conditions, unverified)	1,640, 1,288, 949, 1,322	5,199
C: matched-size real only	Real training images resampled with replacement to B's exact counts (2,029 distinct images drawn; at most 12 repeats); no synthetic images	1,640, 1,288, 949, 1,322	5,199

Why condition C is essential. B has more images than A, so any gain of B over A could come from the larger and better-balanced training set rather than from the synthetic images. C gives a classifier the same number of images and the same class mix as B using only real images. Only a gain of B *over C* would indicate a benefit specific to synthetic data.

3.8 Ethical considerations and evaluation limitations

- **Not a medical device.** No component was tested clinically or reviewed by clinicians. Predictions and generated images must not guide diagnosis or treatment; healthcare use would require clinical validation, regulatory review and oversight that are outside this project.

- **Synthetic images and labels.** Generated images are synthetic and are not patient scans; the class attached to a generated image is the label that was *requested*, not an expert-verified diagnosis. Synthetic images can be mistaken for real ones, so the application labels them clearly.
- **Data and privacy.** The dataset is public and comes without patient identifiers or consent records that this project could inspect; patient-level grouping was therefore impossible. The images are used under the CC BY 4.0 licence shown on the Kaggle page (upstream terms may differ) and are not redistributed.
- **Label noise and bias.** Eight near-duplicate clusters (30 images) carried conflicting class labels, which indicates label noise in the source data. The classes are imbalanced and the data come from a limited set of sources, so performance on other scanners or populations is unknown.
- **Metric limits.** FID and KID use ImageNet-trained Inception features on small grayscale MRI images and do not measure anatomical or clinical validity. The test set is small (492 images, 46 no-tumour), and results are from three classifier seeds and one training run per GAN, with no significance tests.
- **Honest reporting.** Results are reported as measured, including those that do not favour the proposed model.

4. FABRICATION / SIMULATION / PROGRAM OR CODE / DESIGN

This chapter describes the solution that was implemented. It has two parts that are deliberately kept apart: the **research experiments** (Chapters 3 and 5), whose results are saved files, and the **NeuroVision AI web application**, which presents those results and runs two models on request.

4.1 GAN model implementation

The models are written in PyTorch. The research code is organised as follows (all under the project's *src/* and *scripts/* folders):

Table 4.1: Main research code components.

File	Responsibility
data_pipeline.py	Image loading, grayscale/resize/normalise transforms, manifest-based partition dataset
dedup_split.py, dedup_refine.py	Near-duplicate similarity, clustering, quarantine, cluster-disjoint split and the second-stage refinement
dcgan_train.py, wgangp_train.py, cwgangp_train.py	Model definitions, training loops, checkpointing (every 500 updates), sample grids and loss logs; the conditional script also supports resuming
dedup_eval_fid_kid.py	FID/KID evaluation of the three checkpoints against the test partition
dedup_downstream.py	Conditions A, B and C: training, epoch selection on validation, one-time test evaluation, saved predictions
run_dedup_gans.py, run_dedup_stage2.py	Resumable, hash-checked runners that train the GANs and then the downstream and FID/KID stages
verify_dedup_experiment.py	23 automated checks (split integrity, training data, counters, evaluation partitions, preserved earlier results)

4.2 Conditional generation and label conditioning

The conditional generator (class *CondGenerator*) holds an embedding table of four 50-dimensional vectors, one per class in the dataset order glioma = 0, meningioma = 1, no tumour = 2, pituitary = 3. The 100-dimensional noise vector and the selected class embedding are concatenated into a 150-dimensional vector, reshaped to 150×1×1 and passed through five transposed-convolution stages to a 64×64 image (Figure 3.4). The projection critic (*ProjectionCritic*) extracts 512-channel 4×4 features, sums them over space into a 512-dimensional vector h(x), and outputs $\psi(h(x)) + E(y) \cdot h(x)$. During training the gradient penalty is computed with the label held fixed, and no classifier guides the generator. A saved checkpoint stores the generator and critic weights, optimiser states, counters and the class-to-index mapping, which the web application reads to make sure its class order matches.

4.3 Experiment and result-processing pipeline

Each stage writes its outputs, configuration and hashes to *outputs/deduplicated_experiment/*: the split manifest and configuration, per-model checkpoints, metrics, losses and sample grids, FID/KID metrics, the per-run downstream histories and test predictions, and the summary tables. The verification script checks, among other things, that every GAN checkpoint was trained on exactly the manifest's training partition, that every classifier run was evaluated on exactly the test partition, and that earlier experiments were preserved. The report tables in this document are generated from these saved files by script, and sensitivity and specificity were derived from the saved per-run test predictions (Section 5.5) without running any model.

4.4 NeuroVision AI web-application architecture

NeuroVision AI is a two-tier local application (Figure 4.1). The browser interface calls a JSON API; the API validates requests, loads the models on first use, runs inference on the Apple GPU (MPS) or the CPU, and reads the saved result files. The backend never writes to the experiment outputs or the checkpoints.

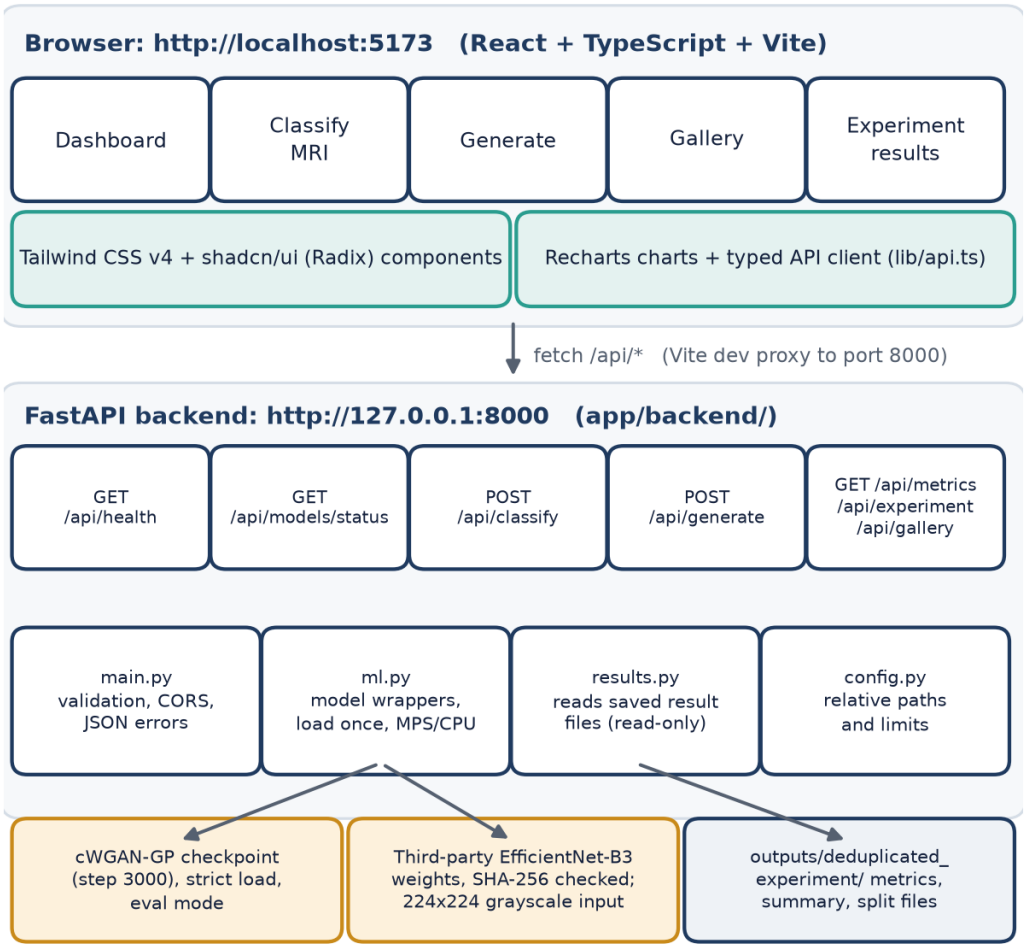


Figure 4.1: Software architecture of NeuroVision AI.

Table 4.2: Technology stack, as recorded in the application's package and requirement files.

Layer	Technology (version)
Frontend framework	React 19.2, TypeScript 6.0, Vite 8.3, React Router 7.18
Styling and components	Tailwind CSS 4.3 with the shadcn/ui component set (Radix primitives, lucide icons)
Charts	Recharts 3.10
Backend API	Python, FastAPI 0.143 with Uvicorn 0.54, Pydantic 2.14
AI inference	PyTorch 2.14.0 and torchvision 0.29.1 (MPS or CPU), Pillow 12.2, NumPy 2.4
Testing	pytest 9.1 with the FastAPI test client; a shell integration smoke test; the TypeScript/Vite production build

4.5 FastAPI backend and PyTorch inference

The backend (folder *app/backend/*) has four modules: *main.py* (routes, validation, CORS and error handling), *ml.py* (model wrappers), *results.py* (reads the saved result files) and *config.py* (paths relative to the project root and limits). The endpoints are:

Table 4.3: API endpoints and their responsibilities.

Endpoint	Responsibility
GET /api/health	Liveness and the research-prototype disclaimer
GET /api/models/status	Device (MPS/CPU), whether the generator and classifier loaded, checkpoint details; states that the project's own A/B/C classifiers are not available
POST /api/classify	Accepts a PNG or JPEG upload; returns the predicted class, softmax probabilities, timing and caveats
POST /api/generate	Body {class, count 1–16, optional seed}; returns synthetic 64×64 PNG images generated by the cWGAN-GP
GET /api/metrics	FID/KID and A/B/C results read from the saved files
GET /api/experiment	Dataset description, methodology, key findings and the documented limitations
GET /api/gallery (+ /image/{id})	A fixed registry of saved figure sheets labelled real, synthetic or mixed; no arbitrary file access

Model and checkpoint loading. Each model is loaded once, on first use, behind a lock, and kept in memory. The generator checkpoint is loaded with *weights_only* and strict key matching, and its stored class mapping is compared with the application's. The classifier weights file must match a recorded SHA-256 digest. If a checkpoint is missing, mismatched or fails to load, the feature is reported as unavailable with an error message; **no substitute model is used and no result is invented**. Seeded generation draws the noise on the CPU so that the same seed and class give the same images on any device.

Table 4.4: Input validation and error handling.

Case	Behaviour
Upload not an image / unsupported format (e.g. GIF, text)	JSON error: not_an_image (415) or unsupported_type (415)
Empty file; file larger than 10 MB	empty_file (400); file_too_large (413)
Image sides below 32 or above 8,192 pixels	bad_dimensions (422)
Unknown class, count outside 1–16, negative seed	Request validation error (422) with the offending field
Model or checkpoint unavailable	classifier_unavailable / generator_unavailable (503) with the reason
Saved result file missing	metrics_unavailable / experiment_unavailable (503)
Cross-origin request from another site	Not allowed: CORS is limited to the local Vite origin (port 5173)

4.6 React + TypeScript + Vite frontend

The frontend (folder *app/frontend/*) is a single-page application with five pages: the **Dashboard** (key results, GAN and classifier charts, model readiness), **Classify MRI** (drag-and-drop or file-picker upload, preview, prediction with class probabilities), **Generate** (class selector, 4–16 images, optional seed, individual and grid PNG download), **Gallery** (saved image sheets labelled real, synthetic or mixed) and **Experiment results** (FID/KID and A/B/C tables and charts, class-wise results, dataset notes and limitations). A navy, teal and off-white theme is defined with Tailwind and shadcn/ui variables; charts use Recharts with error bars; every page shows a research-prototype notice, and the classification and generation pages show the caveats returned by the API. Figures 4.2 to 4.5 are screenshots of the running application captured for this report.

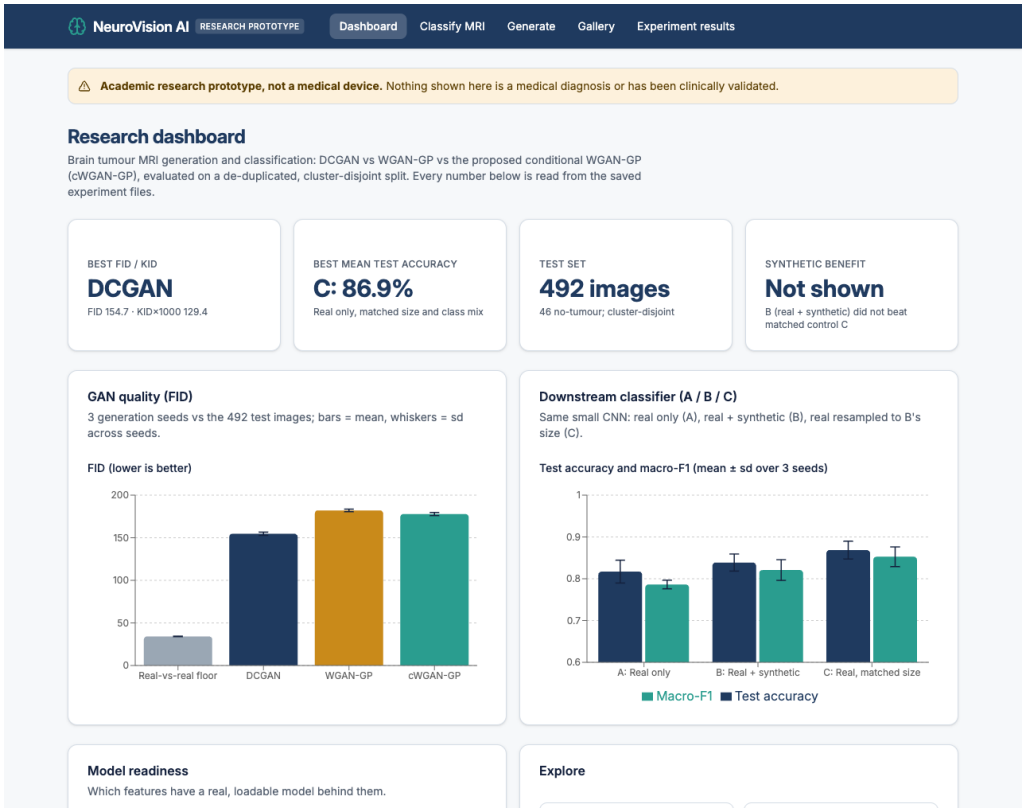


Figure 4.2: NeuroVision AI dashboard (screenshot, top of page): saved FID results, A/B/C chart and key figures.

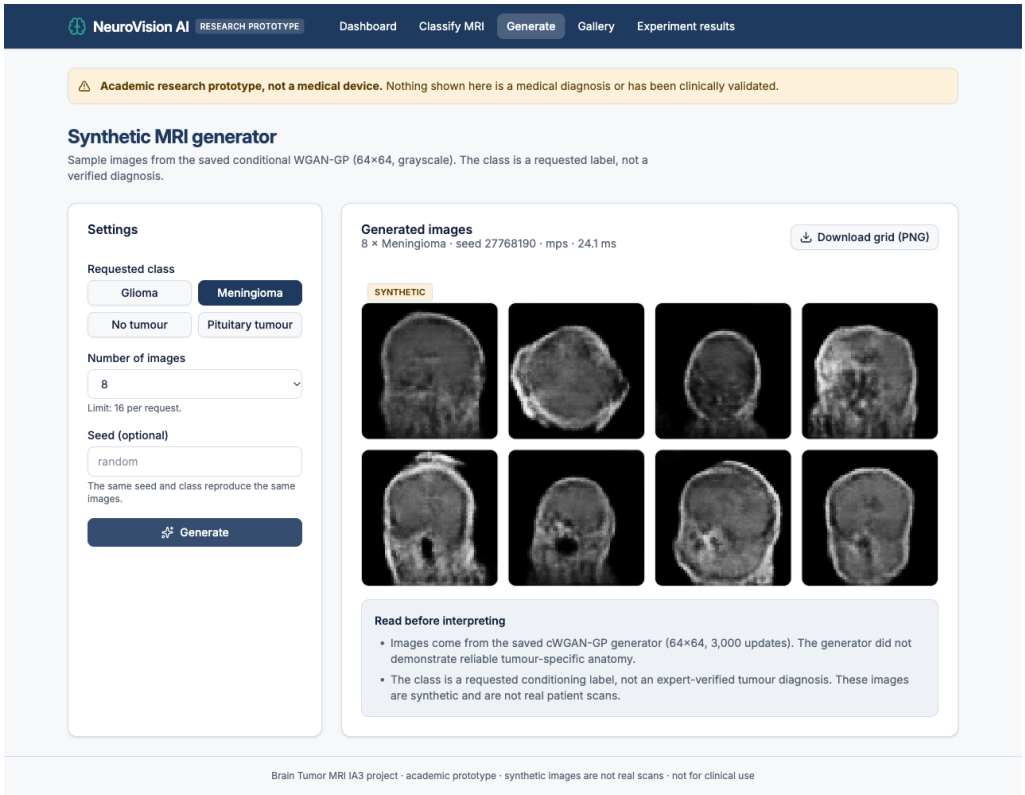


Figure 4.3: Generate page: eight synthetic images for the requested class “Meningioma” (seed shown on screen), marked SYNTHETIC. The class is a requested label, not a diagnosis.

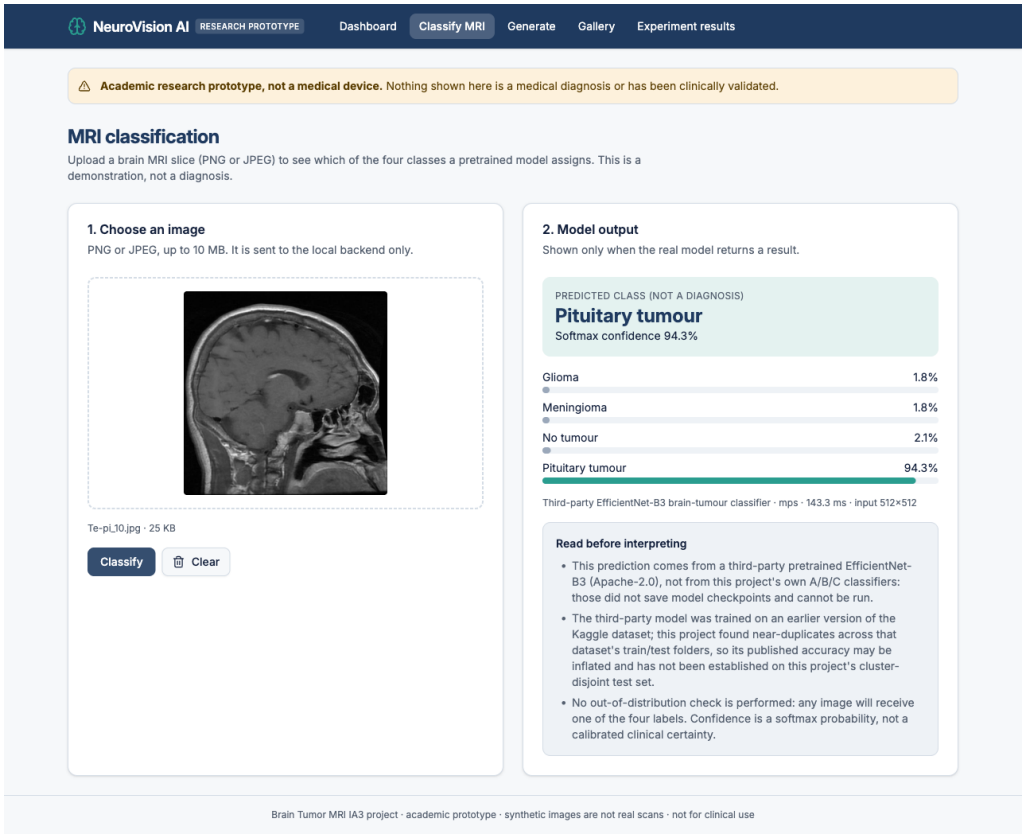


Figure 4.4: Classify page with a dataset test image. The output comes from a third-party pretrained model (Section 4.7), is a demonstration of the application, and is not an evaluation result.

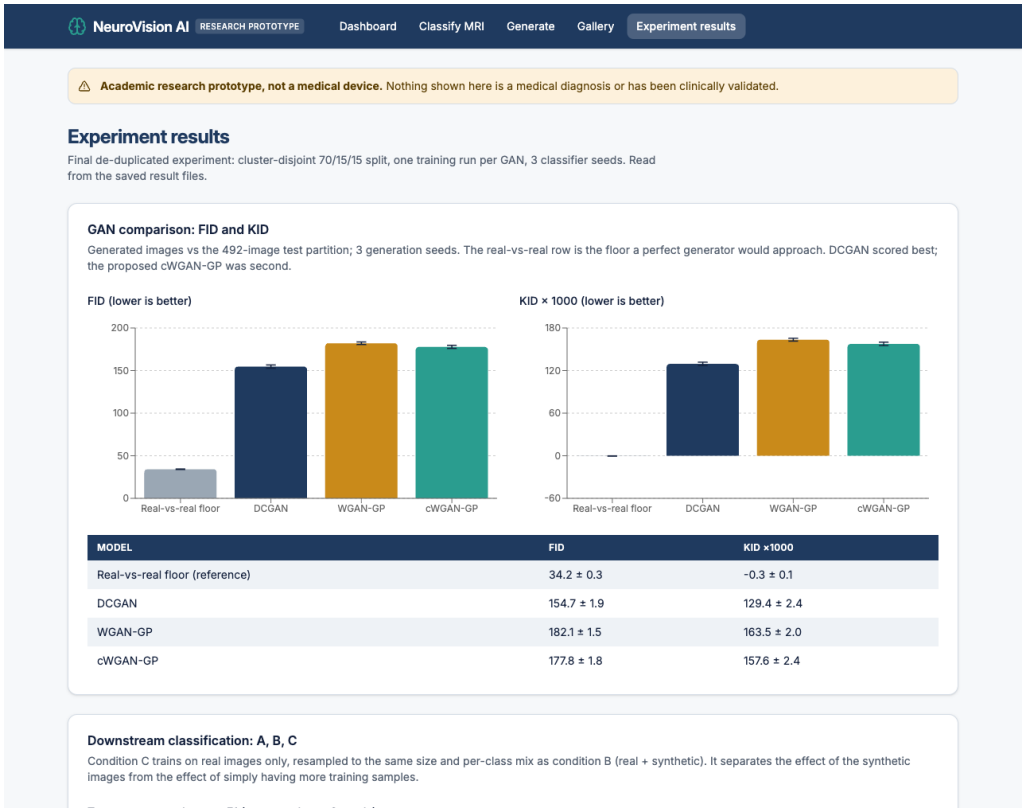


Figure 4.5: Experiment results page (screenshot, top of page): FID and KID charts and table from the saved results.

4.7 Research classifiers versus the application's classifier

Critical distinction

The A, B and C classifiers of the research experiment **did not save model checkpoints**: only their metrics and per-image test predictions were kept, and they were not retrained for the application. They therefore cannot classify a new uploaded image.

The application's *Classify MRI* page uses a **third-party pretrained EfficientNet-B3** [14] from the open-source repository *brain-tumor-classifier* [15] (Apache-2.0 licence; one-channel input resized to 224×224; four output classes in the order glioma, meningioma, no tumour, pituitary). **Its predictions are not results of condition A, B or C.**

That model was trained on an earlier version of the Kaggle dataset. Because this project found near-duplicates across that dataset's train and test folders, its published accuracy may be inflated, and its accuracy on this project's cluster-disjoint test set is **unknown**. The application performs **no out-of-distribution detection**: any image, even one that is not a brain MRI, receives one of the four labels, and the confidence is a softmax probability, not a calibrated clinical certainty.

4.8 Local startup procedure

After the one-time setup (a Python virtual environment for the backend with the pinned requirements, and *npm install* for the frontend), both servers are started from the project root with one command:

./app/run_dev.sh

The backend then listens on http://127.0.0.1:8000 (interactive API documentation at /docs) and the frontend on http://localhost:5173, which is the address opened in the browser. The launcher refuses to start if either port is already in use, and Ctrl+C stops both servers. Configuration (allowed origin, upload limit, optional checkpoint paths) uses environment variables documented in an example file that contains no secrets.

4.9 Tests performed

Table 4.5: Application tests and their recorded outcomes.

Test	What it checks	Recorded outcome
Backend pytest suite (16 tests)	Health and status; both models load with the real checkpoints; metrics endpoint values equal the saved files (including FID 154.7/177.8/182.1 and A/B/C accuracy 0.817/0.839/0.869); generation shape, class dependence and seed reproducibility; invalid input; classification of PNG/JPEG and of local dataset images; error cases; an unavailable model is reported, not substituted; gallery path-traversal blocked; CORS restricted; no hard-coded user paths	16 passed
Integration smoke test (6 checks)	Through the Vite proxy: page served, health, models available, DCGAN FID value, generation of 4 images, rejection of a non-image upload	6/6 passed
Frontend production build	TypeScript type-check and Vite build	Completed without errors
Manual browser checks	Dashboard, classification (upload and result), generation, gallery and results pages in a real browser; the screenshots in this chapter were taken during these checks	Pages rendered and worked as expected
PNG grid download	Download of the generated grid image from the Generate page	Reported as confirmed in the browser by the author; not covered by an automated test

There is **no automated browser-level (end-to-end) test suite**; frontend behaviour was checked manually. Outputs produced during these tests are plumbing checks of the application and are not scientific results.

5. RESULTS AND INTERPRETATION

All numbers in this chapter come from the saved result files of the final de-duplicated experiment (*outputs/deduplicated_experiment/*); sensitivity and specificity were derived from the saved predictions of that experiment. Results of the earlier experiment on the original folders (which contained near-duplicates between training and test data) are superseded and are **not** used or compared here.

5.1 Dataset audit

Table 5.1: Dataset audit and final split.

Quantity	Value
Initial candidate images	7,200
Original Testing images with a near-identical Training copy (audit of the original folders)	1,076 of 1,600
Quarantined images	207
Final usable unique images	3,295
Training / validation / test images	2,311 / 492 / 492
Test images per class (glioma, meningioma, no tumour, pituitary)	196, 122, 46, 128
Patient-level independence	Not proven (no patient identifiers)

The audit shows why the original split could not be trusted: roughly two thirds of its test images had a near-copy in training. The corrected split is much smaller, and the class imbalance is stronger, but its scores are not inflated by copies. Patient-level leakage through different slices of the same patient cannot be excluded.

5.2 GAN generation quality (FID and KID)

Table 5.2: FID and KID (×1000) against the 492 test images; mean ± standard deviation over 3 generation seeds; lower is better.

Model	FID	KID × 1000
Real-vs-real reference floor	34.2 ± 0.3	−0.3 ± 0.1
DCGAN	**154.7 ± 1.9**	**129.4 ± 2.4**
cWGAN-GP	177.8 ± 1.8	157.6 ± 2.4
WGAN-GP	182.1 ± 1.5	163.5 ± 2.0

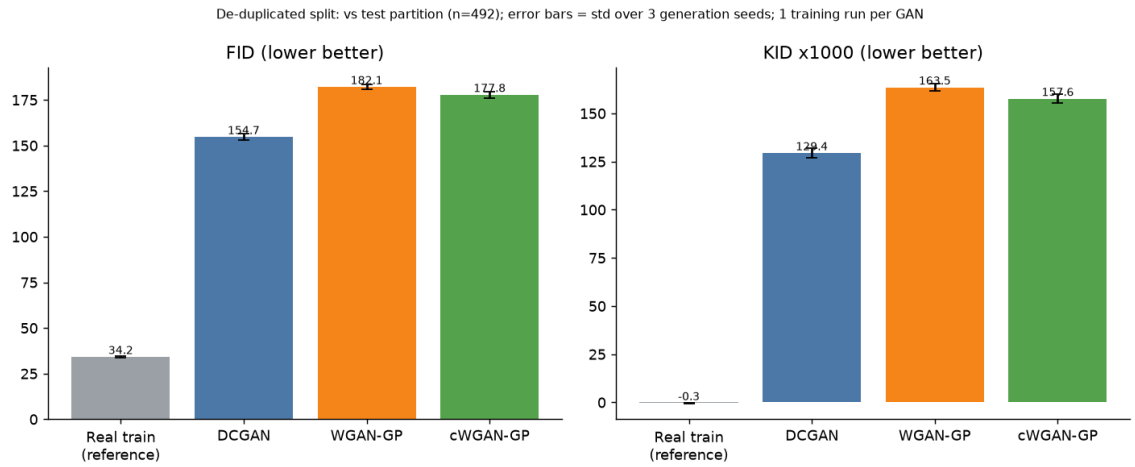


Figure 5.1: FID and KID (×1000) of the three GANs and the real-versus-real floor; error bars show the standard deviation over 3 generation seeds.

DCGAN achieved the best FID and KID of the three generators (154.7 and 129.4), with a wide margin over the other two relative to the seed spread. The proposed cWGAN-GP was second (177.8 and 157.6) and WGAN-GP last (182.1 and 163.5); the 4.3-point FID difference between these two is only a few seed standard deviations and could change across training runs. All three are far above the reference floor of 34.2, i.e. none of the generators matches the real distribution. The comparison is not compute-matched (DCGAN used 3,000 discriminator updates, the WGAN variants 15,000 critic updates), each GAN was trained once, and FID is biased for small samples; the metrics also rely on Inception features learned from colour photographs, so they **do not establish anatomical or clinical validity**.

5.3 Generation findings and class conditioning

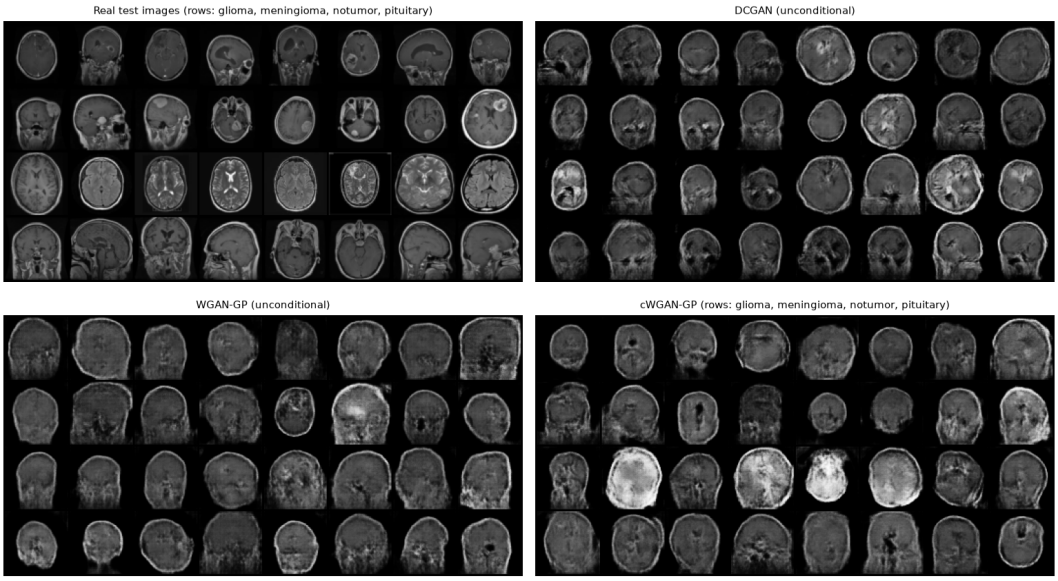


Figure 5.2: Real test images (top left) and samples from DCGAN, WGAN-GP and cWGAN-GP; the rows of the conditional model are glioma, meningioma, no tumour and pituitary.

All three GANs produce head-shaped, MRI-like images with soft internal structure. None shows an identifiable tumour, whereas the real images show clear lesions and fine detail. For the cWGAN-GP the requested class changes the output distribution (the no-tumour images are visibly brighter), but the evidence for *tumour-specific* anatomy is weak, as the per-class analysis shows.

Table 5.3: Per-class FID/KID of the cWGAN-GP (each requested class against the real test images of that class).

Requested class	n generated = n real	FID	KID × 1000	Reliability
Glioma	196	190.9 ± 4.1	157.2 ± 4.2	small for 2,048-d FID
Meningioma	122	213.6 ± 1.1	154.9 ± 0.4	small for 2,048-d FID
No tumour	46	242.1 ± 3.1	166.0 ± 2.0	indicative only (n < 100)
Pituitary	128	239.0 ± 3.0	208.7 ± 4.4	small for 2,048-d FID

Matching each generated class against each real class (KID × 1000), only the meningioma row is closest to its own real class, the glioma and no-tumour labels are the closest generated label for their own real class, and pituitary is not separated. With 46 to 196 images per class, per-class FID is strongly biased and should not be compared across classes. **The generator therefore did not demonstrate reliable tumour-specific anatomy**, and the requested labels are conditioning inputs, not expert-verified diagnoses.

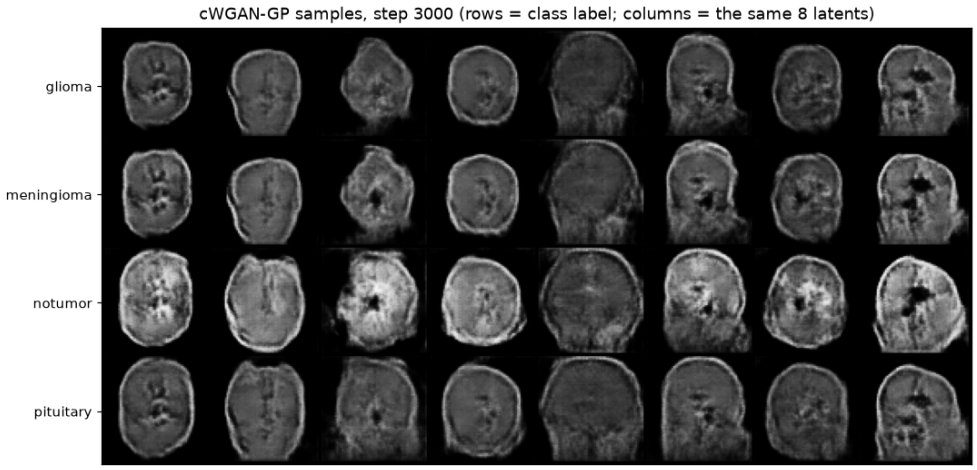


Figure 5.3: cWGAN-GP samples at step 3,000: rows are the requested classes, columns share the same eight noise vectors.

5.4 Downstream classifier comparison (A, B, C)

Table 5.4: Downstream classification on the 492 test images (mean ± standard deviation over 3 seeds).

Condition	Training examples	Test accuracy	Macro-F1	Validation accuracy
A: real only	2,311	0.817 ± 0.027	0.786 ± 0.010	0.812
B: real + synthetic	5,199	0.839 ± 0.020	0.821 ± 0.025	0.827
C: matched-size real only	**5,199**	**0.869 ± 0.021**	**0.853 ± 0.024**	**0.877**

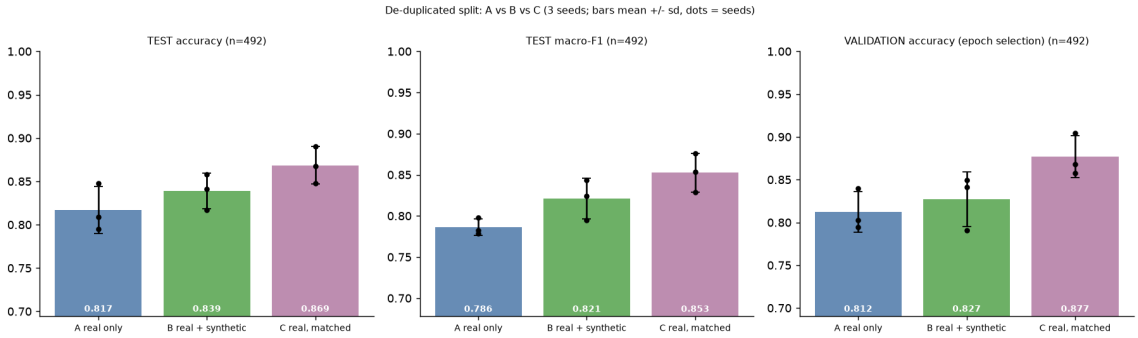


Figure 5.4: Test accuracy, test macro-F1 and validation accuracy of conditions A, B and C (dots are individual seeds).

Table 5.5: Paired per-seed differences (seeds 42, 43, 44).

Difference	Accuracy per seed	Mean	Macro-F1 per seed	Mean
B − A	+0.047, +0.008, +0.010	+0.022	+0.046, +0.012, +0.046	+0.035
C − A	+0.096, +0.059, +0.000	+0.051	+0.097, +0.071, +0.031	+0.066
B − C	−0.049, −0.051, +0.010	−0.030	−0.051, −0.059, +0.015	−0.032

Adding the cWGAN-GP images to the real data (B) beat real-only training (A) in all three seeds (mean accuracy +0.022, macro-F1 +0.035). **However, the matched-size real-only control (C) achieved the highest mean accuracy (0.869) and macro-F1 (0.853)**, scoring above B on average (+0.030 accuracy) and in two of the three seeds. With a 492-image test set (a single run has a 95% sampling interval of about ±3 accuracy points), three seeds and seed standard deviations near 0.02, B and C cannot be separated statistically, and no significance test was run. **The experiment therefore does not demonstrate an advantage of synthetic augmentation beyond the added training volume, and the point estimates favour the control.**

5.5 Class-wise findings, confusion matrices and sensitivity/specificity

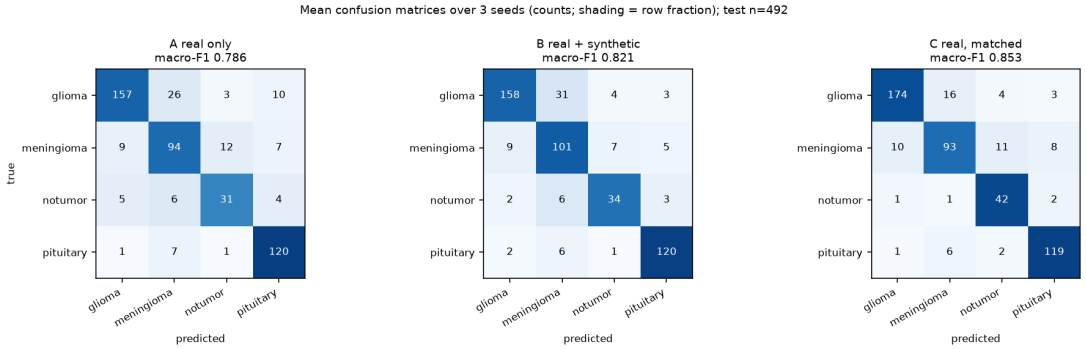


Figure 5.5: Mean confusion matrices over three seeds for conditions A, B and C (counts on the 492 test images).

Table 5.6: Per-class F1 and recall (mean over seeds).

Class (test n)	A: F1 / recall	B: F1 / recall	C: F1 / recall
Glioma (196)	0.85 / 0.80	0.86 / 0.80	0.91 / 0.89
Meningioma (122)	0.74 / 0.77	0.76 / 0.83	0.78 / 0.76
No tumour (46)	0.66 / 0.67	0.74 / 0.75	0.80 / 0.91
Pituitary (128)	0.89 / 0.94	0.92 / 0.93	0.92 / 0.93

The largest confusion in every condition is glioma predicted as meningioma (on average 26, 31 and 16 of 196 glioma images for A, B and C). The no-tumour class, with only 46 test images, is

the least reliable: its F1 rises from 0.66 (A) to 0.74 (B) and 0.80 (C), but each value rests on a handful of images.

Table 5.7: Derived sensitivity and specificity for detecting "any tumour" (glioma, meningioma or pituitary; 446 images) versus "no tumour" (46 images), mean ± sd over 3 seeds.

Condition	Tumour vs no-tumour: sensitivity	Tumour vs no-tumour: specificity
A: real only	0.964 ± 0.021	0.674 ± 0.186
B: real + synthetic	0.973 ± 0.015	0.746 ± 0.153
C: matched-size real only	0.963 ± 0.017	0.906 ± 0.025

These values were computed from the saved test predictions (positive class = any tumour class) and they reproduce the saved accuracies exactly; they are *derived* figures, not separately recorded results. All three conditions detect tumour-bearing images with high sensitivity (about 0.96–0.97), but specificity, i.e. the ability to keep no-tumour images out of the tumour classes, differs: 0.67 for A, 0.75 for B and 0.91 for C. The specificity rests on only 46 no-tumour images, so these numbers are very uncertain and are not clinical performance estimates.

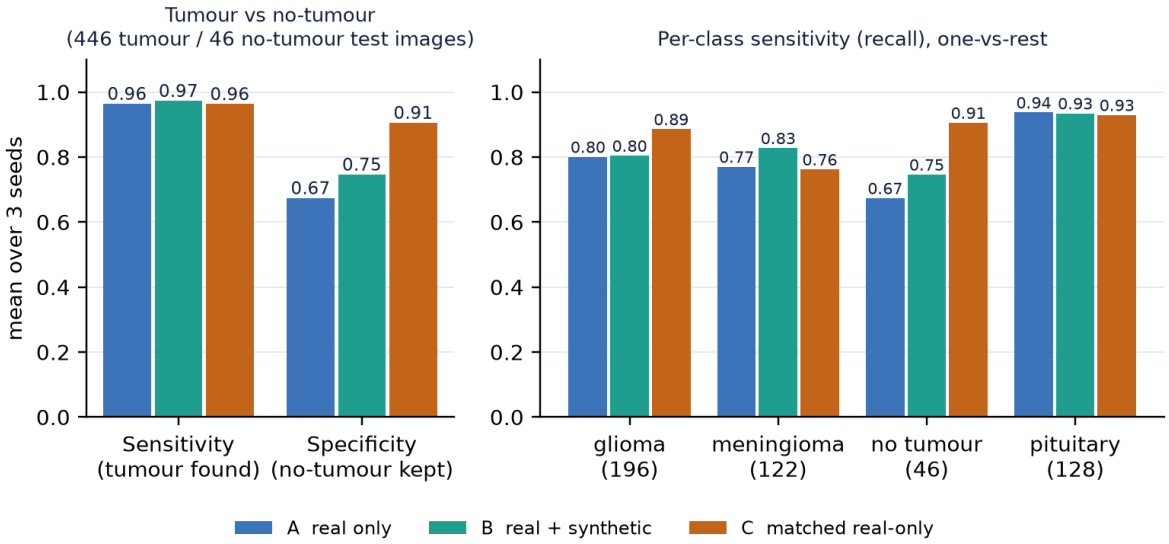


Figure 5.6: Derived sensitivity and specificity (left) and per-class sensitivity (right) for conditions A, B and C, from the saved test predictions.

5.6 Application implementation and testing results

NeuroVision AI was implemented as described in Chapter 4 and tested as listed in Table 4.5: 16 backend tests passed, the 6-check integration smoke test passed, the TypeScript/Vite production build completed, and the dashboard, classification, generation, gallery and results pages were checked manually in a browser (the generated-grid PNG download is reported by the author as confirmed in the browser). No automated browser-level test suite exists. The tests establish that the application runs the real models, serves the saved results unchanged and rejects invalid input;

they do not measure the scientific quality of the generator or of the third-party classifier. In particular, a confident prediction on a test image in the Classify page is a plumbing check and **not evidence of clinical accuracy**.

5.7 What the results support and do not support

Table 5.8: Summary of supported and unsupported statements.

Supported by the results	Not supported
DCGAN gave the lowest FID and KID on this split; cWGAN-GP was second and WGAN-GP last	That the proposed hybrid is the best generator
Adding cWGAN-GP images improved mean accuracy and macro-F1 over real-only training (B over A) in all three seeds	That synthetic images improve classification beyond added training volume
A size- and class-mix-matched real-only control scored at least as well on average	That the generated labels correspond to real tumour classes
The requested label changes the generated distribution (clearest for no tumour)	That the images are anatomically or clinically meaningful
The split is protected against copy-level leakage (0 clusters cross partitions)	That the result is free of patient-level leakage
The application runs the real models and shows the saved results	That any ranking persists across other training seeds, longer training or other hyperparameters; or any estimate of performance on new patients

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This micro project audited a public brain-tumour MRI dataset, found that its original test images were not independent of the training images, and rebuilt a cluster-disjoint split of 3,295 unique images. On that split three GANs were compared. **DCGAN achieved the best generation metrics** (FID 154.7, KID×1000 129.4), the proposed conditional WGAN-GP was second (177.8, 157.6) and WGAN-GP third (182.1, 163.5). In the downstream experiment, adding synthetic images raised the mean test accuracy over real-only training (0.817 to 0.839), but **the matched-size real-only control achieved the best mean accuracy (0.869) and macro-F1 (0.853)**, so the study does not demonstrate a benefit of synthetic augmentation beyond added training volume. The generator produced MRI-like images but did not demonstrate reliable tumour-specific anatomy.

The work was packaged as NeuroVision AI, a tested React + TypeScript and FastAPI + PyTorch application that presents the saved results, generates synthetic samples with the saved cWGAN-GP checkpoint and demonstrates a third-party pretrained classifier, with the research limits displayed in the interface. The project demonstrates careful implementation, fair comparison and honest reporting; it does not show that the proposed model outperforms the baselines, and it is not a clinically validated system.

6.2 Limitations

- Small, imbalanced test set (492 images; 46 no-tumour); no significance tests; three classifier seeds and a single training run per GAN.
- Patient-level independence is unproven; near-duplicate detection is heuristic and may miss rescaled or re-cropped copies. The dataset download record is unverified and its upstream sources may have separate terms.
- FID/KID use ImageNet Inception features on 64×64 grayscale MRI; short GAN training (3,000 updates) that is not compute-matched between DCGAN and the WGAN variants; the cWGAN-GP run was interrupted at step 2,861 and resumed (cause unconfirmed).
- Synthetic labels are requested conditions, not verified diagnoses; weak evidence for tumour-specific generation.
- The application's classifier is third-party, may have inflated published accuracy, has unknown accuracy on this project's test set, and has no out-of-distribution detection; the application has no automated browser-level tests.

6.3 Future scope

- Use larger and more diverse datasets, ideally with patient identifiers so that splits can be made at patient level.
- Run more independent GAN training runs and classifier seeds, with significance testing and compute-matched baselines and longer training.
- Improve conditional generation (for example stronger conditioning, larger resolution, or expert-reviewed labels) and evaluate class consistency of the generated images.
- Evaluate medical-image structure directly, for example with expert review or domain-trained feature extractors instead of ImageNet features alone.
- Save a research classifier checkpoint (for example from condition C) if new-image inference with the research classifier is needed, and evaluate it on the cluster-disjoint test set.
- Add automated browser-level tests and out-of-distribution handling to NeuroVision AI.
- Before any healthcare deployment: clinical validation, regulatory review, privacy assessment and continuous clinical oversight.

REFERENCES

[1] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville and Y. Bengio, “Generative Adversarial Networks,” arXiv:1406.2661, 2014.

[2] M. Mirza and S. Osindero, “Conditional Generative Adversarial Nets,” arXiv:1411.1784, 2014.

[3] A. Radford, L. Metz and S. Chintala, “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks,” arXiv:1511.06434, 2015 (submitted to ICLR 2016).

[4] M. Arjovsky, S. Chintala and L. Bottou, “Wasserstein GAN,” arXiv:1701.07875, 2017.

[5] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin and A. Courville, “Improved Training of Wasserstein GANs,” arXiv:1704.00028, 2017.

[6] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*; arXiv:1706.08500.

[7] M. Bińkowski, D. J. Sutherland, M. Arbel and A. Gretton, “Demystifying MMD GANs,” in *International Conference on Learning Representations (ICLR)*, 2018; arXiv:1801.01401.

[8] T. Miyato and M. Koyama, “cGANs with Projection Discriminator,” in *International Conference on Learning Representations (ICLR)*, 2018; arXiv:1802.05637.

[9] United Nations, “Transforming our world: the 2030 Agenda for Sustainable Development,” 2015. https://sdgs.un.org/2030agenda

[10] Karunya Institute of Technology and Sciences, official website (mission statement and focus areas). https://www.karunya.edu/ and https://kcode.karunya.edu/about-us (accessed 27 October 2026).

[11] M. Nickparvar, “Brain Tumor MRI Dataset,” Kaggle, Version 2, licence CC BY 4.0. https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset (accessed October 2026). Compiled from the figshare, SARTAJ and Br35H datasets.

[12] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *International Conference on Learning Representations (ICLR)*, 2015; arXiv:1412.6980.

[13] torch-fidelity (version 0.4.0), open-source package for FID, KID and Inception score. https://github.com/toshas/torch-fidelity

[14] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *International Conference on Machine Learning (ICML)*, 2019; arXiv:1905.11946.

[15] Phi-007, “brain-tumor-classifier,” open-source repository (Apache-2.0), EfficientNet-B3 brain MRI classifier. https://github.com/Phi-007/brain-tumor-classifier

APPENDIX A. SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

This appendix is the Software Requirements Specification of **NeuroVision AI**, version 1.0. It follows the section structure of IEEE Std 830-1998 (and ISO/IEC/IEEE 29148). It describes the application as it was built and tested in Chapter 4, not a future system: every requirement can be traced to a page, an API endpoint or a test (Table A.7).

A.1 Introduction

A.1.1 Purpose

The SRS states what NeuroVision AI shall do and the qualities it shall have. Its readers are the course evaluators, the developer and anyone who later extends or re-runs the project. It is the reference for the design in Appendix B.

A.1.2 Scope

NeuroVision AI is a local, two-tier web application (React + TypeScript frontend, FastAPI + PyTorch backend) that (i) presents the saved results of the de-duplicated GAN experiment, (ii) generates synthetic 64×64 grayscale brain-MRI slices for a requested class with the saved cWGAN-GP checkpoint, and (iii) demonstrates a third-party pretrained classifier on an uploaded image. It corresponds to objective O6 of Section 1.3.

Out of scope: training or re-training any model from the interface; user accounts and log-in; storing uploaded images or any patient data; clinical decision support; cloud hosting. The application is **not a medical device** and has not been clinically validated.

A.1.3 Definitions, acronyms and abbreviations

Table A.1: Definitions, acronyms and abbreviations.

Term	Meaning
SRS	Software Requirements Specification.
API	Application Programming Interface; here the JSON/HTTP interface of the FastAPI backend (Table 4.3).
cWGAN-GP	Conditional Wasserstein GAN with gradient penalty and a projection critic (the proposed model, Section 3.4).
FID / KID	Fréchet / Kernel Inception Distance, the generation-quality scores of Section 3.6.
Conditions A, B, C	Downstream classifier training sets: real only (A), real + synthetic (B), size-matched real only (C).
Checkpoint	A saved file of trained model weights (for the generator, step 3,000 of the cWGAN-GP run).
Synthetic image	An image produced by the generator; not a patient scan. Its class is a *requested* label, not a diagnosis.
Softmax probability	The classifier's normalised output score per class; not a calibrated clinical certainty.

Term	Meaning
MPS	Metal Performance Shaders, the Apple-GPU backend of PyTorch used when available (otherwise the CPU is used).
CORS	Cross-Origin Resource Sharing; here restricted to the local Vite origin.
SHA-256	Cryptographic digest used to check the classifier weights file.
SPA	Single-page application (the React frontend).

A.1.4 References

- IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*; ISO/IEC/IEEE 29148:2018, *Requirements engineering* (structure of this appendix).
- Chapters 3 to 5 of this report (methodology, implementation, results), and the references [1]–[15] listed on page 31.

A.1.5 Overview of this appendix

Section A.2 describes the product and its users in general terms. Section A.3 lists the external interfaces, the 15 functional requirements (FR-01 to FR-15), the 9 non-functional requirements (NFR-01 to NFR-09) and the traceability of each functional requirement to the implementation and its test.

A.2 Overall Description

A.2.1 Product perspective

NeuroVision AI is a self-contained product. It does not call any external service while running. It reads three kinds of files that were produced earlier by the research code: the saved experiment results (*outputs/deduplicated_experiment/*), the cWGAN-GP checkpoint and the third-party classifier weights. The backend never writes to these files. The architecture is shown in Figure 4.1 and refined in Appendix B.

A.2.2 Product functions

- Show a dashboard of the key saved results and the readiness of each model.
- Show the full experiment results (FID/KID, conditions A/B/C, class-wise findings, dataset notes, limitations).
- Classify an uploaded PNG/JPEG brain-MRI slice with the third-party EfficientNet-B3 (demonstration).
- Generate and download synthetic 64×64 MRI images for a requested class with the saved cWGAN-GP.
- Browse a fixed gallery of saved figure sheets labelled real, synthetic or mixed.

- Report service health and model availability, and state the research-prototype status on every page.

A.2.3 User classes and characteristics

Table A.2: User classes.

User class	Characteristics	Main functions used
Researcher / student (primary)	Knows what a GAN and an MRI slice are; uses the local application to explore results and models; no programming needed.	All five pages
Evaluator / faculty reviewer	Reads results and tries the demonstrations during the presentation; needs the limits stated clearly.	Dashboard, Experiment results, Gallery, Classify, Generate
Developer / maintainer	Starts the servers with *run_dev.sh*, runs the tests, sets environment variables.	Health and status endpoints, tests, configuration

A.2.4 Operating environment

Table A.3: Operating environment.

Item	Environment (as recorded in Tables 3.4 and 4.2)
Client	A current desktop web browser; the interface is served at http://localhost:5173.
Server	Python 3.14.3 with FastAPI 0.143, Uvicorn 0.54, Pydantic 2.14, PyTorch 2.14.0, torchvision 0.29.1, Pillow 12.2, NumPy 2.4; listens on http://127.0.0.1:8000.
Frontend build	React 19.2, TypeScript 6.0, Vite 8.3, React Router 7.18, Tailwind CSS 4.3 with shadcn/ui, Recharts 3.10.
Hardware used	Apple MacBook Air (M4 chip, 16 GB unified memory), macOS; the Apple GPU (MPS) is used when available, otherwise the CPU.

A.2.5 Design and implementation constraints

- The application runs locally; both servers bind to the local machine and CORS allows only the local Vite origin.
- The generator produces 64×64 grayscale images only; the classifier input is resized to 224×224 grayscale.
- Uploads are limited to PNG or JPEG, at most 10 MB, with sides between 32 and 8,192 pixels; a request may generate 1 to 16 images.
- The third-party classifier is used under its Apache-2.0 licence; the dataset is not redistributed (Section 3.1).
- No model may be substituted for another and no result may be invented when a model or a result file is missing.

A.2.6 Assumptions and dependencies

- The cWGAN-GP checkpoint (step 3,000) and the classifier weights file (with its recorded SHA-256 digest) are present.
- The saved result files of the final de-duplicated experiment are present and unchanged.
- The Python environment and the frontend packages have been installed once, as described in Section 4.8.

A.3 Specific Requirements

A.3.1 External interface requirements

Table A.4: External interface requirements.

Interface	Requirement
User interface	A single-page web interface with five pages (Dashboard, Classify MRI, Generate, Gallery, Experiment results), a common header with navigation and a research-prototype notice on every page (Section B.4).
Hardware interface	No special hardware. The Apple GPU (MPS) is used if present, otherwise the CPU.
Software interface	Frontend and backend libraries listed in Table A.3; PyTorch checkpoints loaded with *weights_only* and strict key matching.
Communication interface	JSON over HTTP between the browser (through the Vite proxy on port 5173) and the backend on port 8000; the upload uses *multipart/form-data*; endpoints as in Table 4.3; errors as *{error: {code, message}}* (Table 4.4).

A.3.2 Functional requirements

Priority is High (needed for the stated purpose of the application) or Medium (useful, not essential). Each requirement is verifiable; Table A.7 states how.

Table A.5: Functional requirements.

ID	The system shall …	Priority
FR-01	accept a brain-MRI image as a PNG or JPEG file by drag-and-drop or the file picker, and show a preview before it is submitted.	High
FR-02	validate every uploaded file, in the browser and on the server: type (PNG/JPEG), not empty, at most 10 MB, sides between 32 and 8,192 pixels, decodable as an image; a rejected file shall receive a specific error code and message.	High
FR-03	classify a validated image into glioma, meningioma, no tumour or pituitary tumour with the third-party EfficientNet-B3, and return the predicted class, the probability of all four classes, the inference time and the device.	High
FR-04	show with every classification result the probability bars and the caveats (third-party model, not the A/B/C classifiers, no out-of-distribution check, not a diagnosis).	High
FR-05	generate 1 to 16 synthetic 64×64 grayscale MRI images for a requested class (glioma, meningioma, no tumour, pituitary) with the saved cWGAN-GP checkpoint.	High

ID	The system shall …	Priority
FR-06	accept an optional integer seed (0 to $2^{31}-1$) and return the same images for the same seed and class on any device.	Medium
FR-07	mark every generated image as SYNTHETIC and state that the class is a requested label, not an expert-verified diagnosis.	High
FR-08	let the user download each generated image and the whole grid as PNG files.	Medium
FR-09	show a dashboard with the best FID/KID, the best mean test accuracy, the test-set size and the synthetic-benefit finding, with the FID and A/B/C charts.	High
FR-10	show an experiment-results page with the FID/KID table and charts, the A/B/C accuracy and macro-F1, the class-wise F1, the dataset and methodology notes and the documented limitations, all read unchanged from the saved result files.	High
FR-11	provide a gallery of saved figure sheets, each labelled real, synthetic or mixed, served only from a fixed registry of known files.	Medium
FR-12	report service health, and the device, the availability of the generator and the classifier, checkpoint details and, when unavailable, the reason.	High
FR-13	load each model once, on first use, verify it (strict key match, class-order check, SHA-256 of the classifier weights) and report a model that fails these checks as unavailable instead of using a substitute.	High
FR-14	display a research-prototype notice on every page and include a disclaimer in the classification, generation and health responses.	High
FR-15	return every failure as a structured JSON error and show its message to the user without stopping the application, including the case that the backend cannot be reached.	High

A.3.3 Non-functional requirements

Table A.6: Non-functional requirements.

ID	Category	The system shall …	Verification
NFR-01	Performance	answer a classification request, and a request for 16 generated images, within 5 seconds on the reference hardware once the models are loaded (the first request also pays the one-time load).	Observed 143.3 ms (classification) and 24.1 ms (8 images) in Figures 4.3 and 4.4; no load test was run.
NFR-02	Security	accept only PNG/JPEG uploads up to 10 MB, load checkpoints with *weights_only*, allow cross-origin calls only from the local Vite origin, serve gallery images only from a fixed registry and contain no hard-coded user paths.	pytest: CORS, gallery path-traversal and hard-coded-path tests; error-handling test.
NFR-03	Privacy and data protection	process an uploaded image in memory only: it is not written to disk and not sent to any external service, and its content is not logged.	Code inspection of *main.py* (bytes read into memory, never saved); the page states that the image goes to the local backend only.

ID	Category	The system shall …	Verification
NFR-04	Reliability	keep running when a model or result file is missing or fails to load, report the affected feature as unavailable with the reason, and never substitute another model or invent a result.	pytest: unavailable-model test; metrics equal the saved files.
NFR-05	Accuracy of presented results	show metric values equal to those in the saved result files, the only arithmetic being the ×1000 scaling of KID.	pytest: metrics-match-saved-files test; smoke test (DCGAN FID 154.7).
NFR-06	Usability and accessibility	be usable with keyboard and screen reader: labelled navigation, an upload area operable with Enter or Space, labelled form controls, announced results, inline validation of the seed, and a visible busy state while a model runs.	Manual browser checks (Table 4.5); no automated accessibility audit was run.
NFR-07	Safety and ethics	never present an output as a diagnosis: research status, synthetic status and the limits of each model shall be visible wherever a result is shown.	Manual inspection of every page (Figures 4.2 to 4.5).
NFR-08	Portability	run on macOS with the Apple GPU or on the CPU, with all paths relative to the project root and configurable through environment variables.	pytest: no-hard-coded-paths test; the models ran on MPS (device shown in the screenshots).
NFR-09	Maintainability and testability	keep routes, model wrappers, result readers and configuration in separate modules, use a typed API client, and keep an automated test suite.	16 backend tests, 6-check smoke test and a type-checked production build all passed (Table 4.5).

A.3.4 Requirements traceability

Table A.7: Traceability of the functional requirements to the implementation and the tests.

ID	Implemented in	Verification (Table 4.5)
FR-01	Classify page (drop zone, file picker, preview)	Manual browser check
FR-02	Classify page checks; *POST /api/classify*	*test_classify_error_handling*; smoke test (text file rejected)
FR-03	*POST /api/classify*; ClassifierService.predict	*test_classify_valid_png_and_jpeg*; *test_classify_real_dataset_images_if_present*
FR-04	Classify page; *caveats* field of the response	Manual browser check (Figure 4.4)
FR-05	*POST /api/generate*; GeneratorService.generate	*test_generate_shape_class_and_seed_reproducible*; smoke test (4 images)
FR-06	Seed field; seeded CPU noise in GeneratorService	*test_generate_shape_class_and_seed_reproducible*
FR-07	Generate page (SYNTHETIC badge, notes); *synthetic* and *caveats* fields	Manual browser check (Figure 4.3)
FR-08	Generate page download buttons (single image, grid)	Author's browser check; no automated test
FR-09	Dashboard page; *GET /api/metrics*	*test_metrics_match_saved_files*; manual check (Figure 4.2)

ID	Implemented in	Verification (Table 4.5)
FR-10	Experiment results page; *GET /api/metrics*, */api/experiment*	*test_experiment_lists_limitations*; manual check (Figure 4.5)
FR-11	Gallery page; *GET /api/gallery*, */api/gallery/image/{id}*	*test_gallery_registry_only*
FR-12	*GET /api/health*, */api/models/status*	*test_health*; *test_status_models_load*; smoke test
FR-13	*_Service.ensure*; GeneratorService and ClassifierService *_load*	*test_unavailable_model_is_reported_not_substituted*
FR-14	Layout notice on every page; *disclaimer* in responses	*test_health*; manual inspection
FR-15	Exception handlers and *err()* in *main.py*; *ApiError* in *api.ts*	*test_generate_rejects_bad_input*; *test_classify_error_handling*

Scope of the verification
The tests show that the application behaves as specified (it runs the real models, serves the saved results unchanged and rejects invalid input). They do not measure the scientific quality of the generator or of the third-party classifier, and there is no automated browser-level test suite (Section 4.9).

APPENDIX B. DESIGN DOCUMENT

This appendix is the design of NeuroVision AI: the overall design and its decisions (B.1), five UML diagrams (B.2), the algorithms behind the data preparation, the models and the application (B.3) and the user-interface design (B.4). It realises the requirements of Appendix A; the implementation is described in Chapter 4.

B.1 Design Overview

B.1.1 Design principles

- **Truthful presentation.** A result is read from a saved file or produced by a real, verified model. If neither is possible the feature is reported unavailable (FR-13, NFR-04).
- **Read-only application.** The backend only reads the experiment outputs and checkpoints; the research experiments stay reproducible and unchanged.
- **Validate at the boundary.** Every request is checked (type, size, dimensions, class, count, seed) before any model runs.
- **Research limits are part of the interface.** Notices, badges and caveats are produced by the backend and shown wherever a result appears.

B.1.2 Architecture and design decisions

The architecture is the two-tier design of Figure 4.1: a React single-page application calls a FastAPI backend whose four modules are *main.py* (routes, validation, error handling), *ml.py* (model services), *results.py* (saved-result readers) and *config.py* (paths and limits). Table B.1 records the main design decisions and their reasons.

Table B.1: Main design decisions.

Decision	Reason
Each model is loaded lazily, once, behind a lock (*_Service.ensure*).	Fast start-up; safe when several requests arrive together; a load error is stored and reported, not retried silently.
Checkpoints are loaded with *weights_only=True* and strict key matching; the generator's class mapping and the classifier's SHA-256 are checked.	Prevents loading unexpected code or a mismatched model (NFR-02, FR-13).
Latent noise for a seeded request is drawn on the CPU.	The same seed and class give the same images on any device (FR-06).
Generated images are returned inside the JSON response as base64 PNG data URIs.	One request, no image files to store or clean up; nothing is written to disk (NFR-03).
The gallery serves files only from a fixed registry of identifiers.	No file path comes from the user, so path traversal is impossible (NFR-02).
The classifier is the third-party EfficientNet-B3; the A/B/C classifiers are not used.	Conditions A, B and C saved no checkpoints (Section 4.7); the limitation is stated on the page.

Decision	Reason
Errors use one JSON shape *{error: {code, message}}* with an HTTP status.	The client can show a precise message for every case in Table 4.4 (FR-15).
The Vite dev server proxies */api* to the backend; CORS allows only the Vite origin.	The browser sees one origin; no other website can call the local API.

B.1.3 Data and file design

There is no database and no user data is stored. The application reads the files of Table B.2, all under the project root.

Table B.2: Files read by the application.

File (under outputs/deduplicated_experiment/ unless stated)	Content	Used for
gan_comparison/metrics.json	FID and KID per model and generation seed; per-class cWGAN-GP scores	Dashboard, Experiment results (GET /api/metrics)
downstream/summary.json	Accuracy, macro-F1, confusion matrices and paired differences of conditions A, B, C	Dashboard, Experiment results
dedup_audit_summary.json, split_config.json, split_manifest.csv	Audit counts, split settings, one row per image with its partition	Dataset notes (GET /api/experiment)
gan/gan_verification.json	Generator updates and run time per GAN	Experiment notes
gan/cwgangp/checkpoint_step_3000.pt h	cWGAN-GP generator and critic weights, configuration, class mapping	Generator service
external_models/…/weights/best_mode l.pth (project root)	Third-party EfficientNet-B3 weights (SHA-256 recorded)	Classifier service
outputs/submission/figures/*.png and gan/*/samples_step_3000.png	Saved figure sheets	Gallery (fixed registry)

B.2 UML Diagrams

Five UML diagrams describe the system: a use-case diagram (Figure B.1), a class diagram (Figure B.2), two sequence diagrams (Figures B.3 and B.4) and an activity diagram (Figure B.5). The component and deployment view is the architecture diagram of Figure 4.1.

B.2.1 Use-case diagram

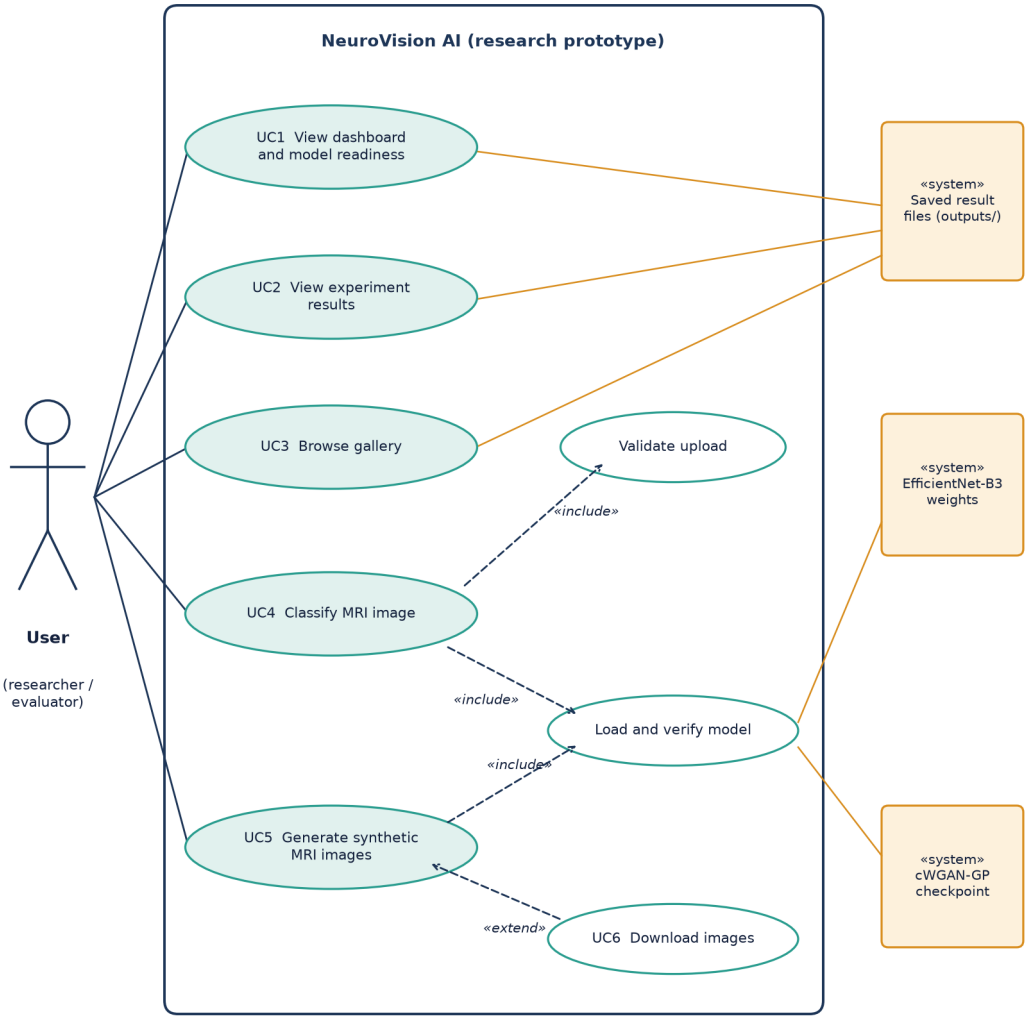


Figure B.1: Use-case diagram of NeuroVision AI. The user has six use cases; validation and model loading are included behaviours, downloading extends generation.

Two use cases are specified in full, because they are the ones that run a model.

Table B.3: Use-case specification of UC4.

Field	UC4: Classify MRI image
Actors	User (primary); EfficientNet-B3 weights (secondary, system).
Precondition	Backend running; classifier weights present and their SHA-256 matches.
Main flow	1. The user chooses or drops a PNG/JPEG image. 2. The page checks type and size and shows a preview. 3. The user clicks Classify. 4. The page sends the file to *POST /api/classify*. 5. The server validates size, content, format and dimensions. 6. The server converts the image (grayscale, 224×224, normalised) and runs the model. 7. The page shows the predicted class, the four probabilities, time, device and caveats.

Field	UC4: Classify MRI image
Alternative flows	A1: wrong type or size in the browser: message shown, no request. A2: the server rejects the file: JSON error 400, 413, 415 or 422 shown. A3: model unavailable: error 503 and the button is disabled with the reason.
Postcondition	Result displayed; the image is not stored.
Business rule	The output is labelled a demonstration, not a diagnosis.

Table B.4: Use-case specification of UC5.

Field	UC5: Generate synthetic MRI images (UC6 extends it)
Actors	User (primary); cWGAN-GP checkpoint (secondary, system).
Precondition	Backend running; checkpoint present, its class mapping equal to the application's.
Main flow	1. The user selects a class, a count (1 to 16) and optionally a seed. 2. The page validates the seed. 3. The user clicks Generate. 4. The page sends *POST /api/generate*. 5. The server validates the body, draws seeded noise, runs the generator with the class index and encodes the images as PNG. 6. The page shows the images with the SYNTHETIC badge, the seed used and the caveats.
Alternative flows	A1: invalid class, count or seed: error 422 shown. A2: generator unavailable: error 503 with the reason. UC6: the user downloads one image or the grid as PNG (built in the browser).
Postcondition	Images displayed and, if chosen, saved by the user; nothing is stored by the server.
Business rule	The requested class is a conditioning label, not a verified tumour type.

B.2.2 Class diagram

_Service holds what both model services share (lock, loaded flag, stored error, device and *ensure()*, which loads the model once). *GeneratorService* and *ClassifierService* specialise it and wrap *CondGenerator* and *BrainTumorEfficientNet*, whose definitions are copied from the research code and the third-party repository. *main* and *results* are modules of functions, shown as stereotyped classes.

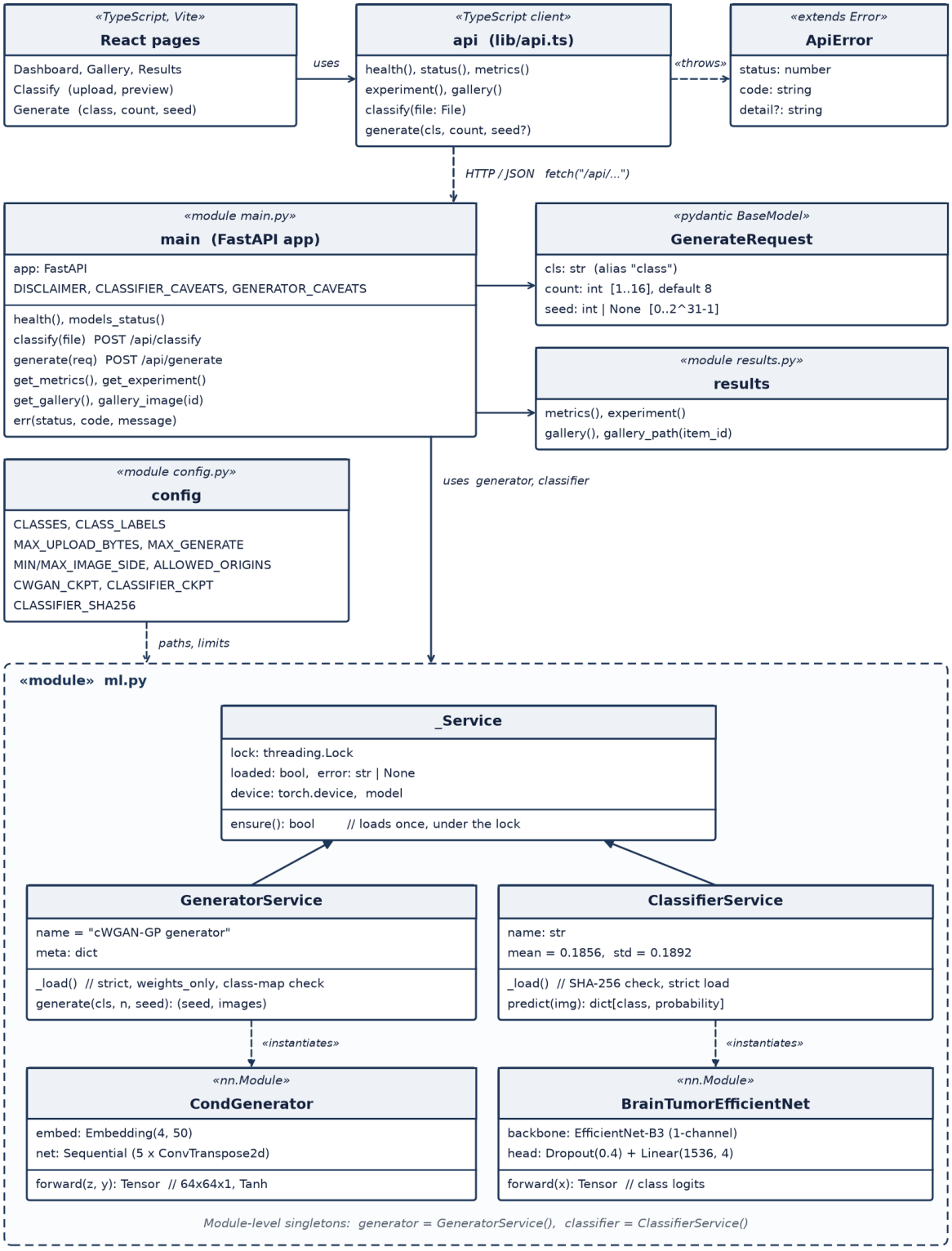


Figure B.2: Class diagram: frontend client, FastAPI module, request model, result readers and the model services of ml.py with the two networks they instantiate.

B.2.3 Sequence diagrams

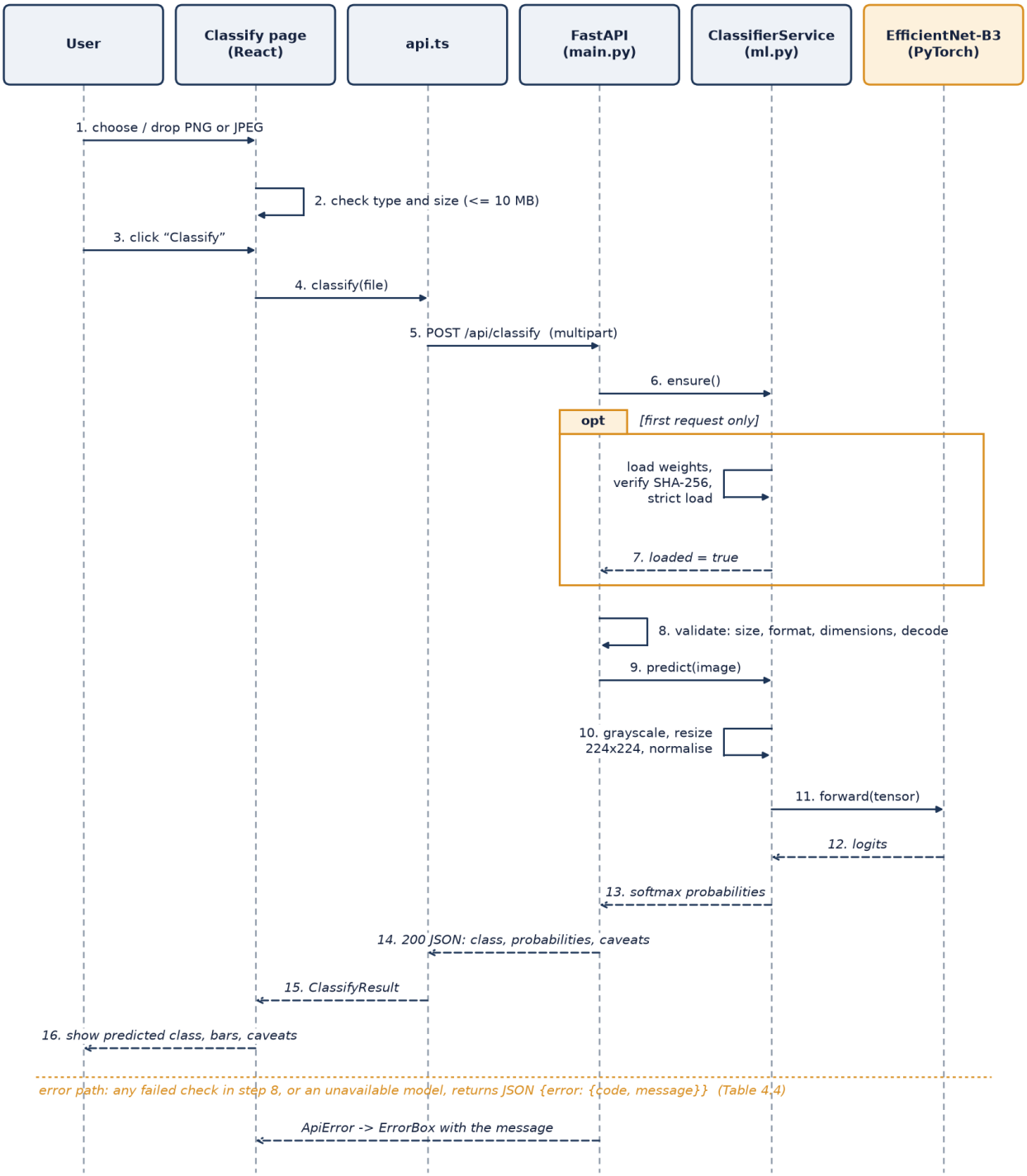


Figure B.3: Sequence diagram of the classification request, including the one-time model load and the error path.

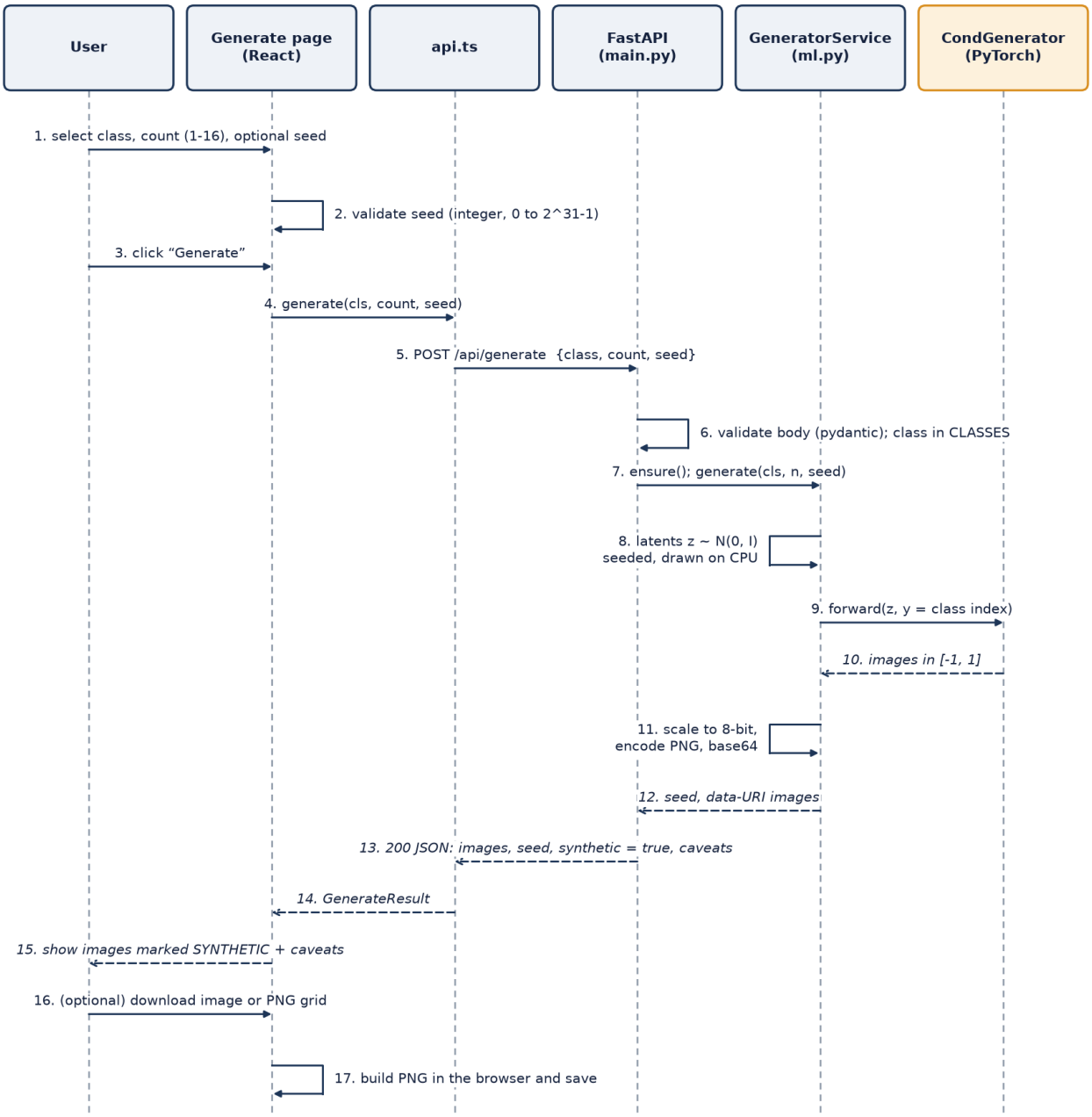


Figure B.4: Sequence diagram of the generation request and the optional download.

B.2.4 Activity diagram

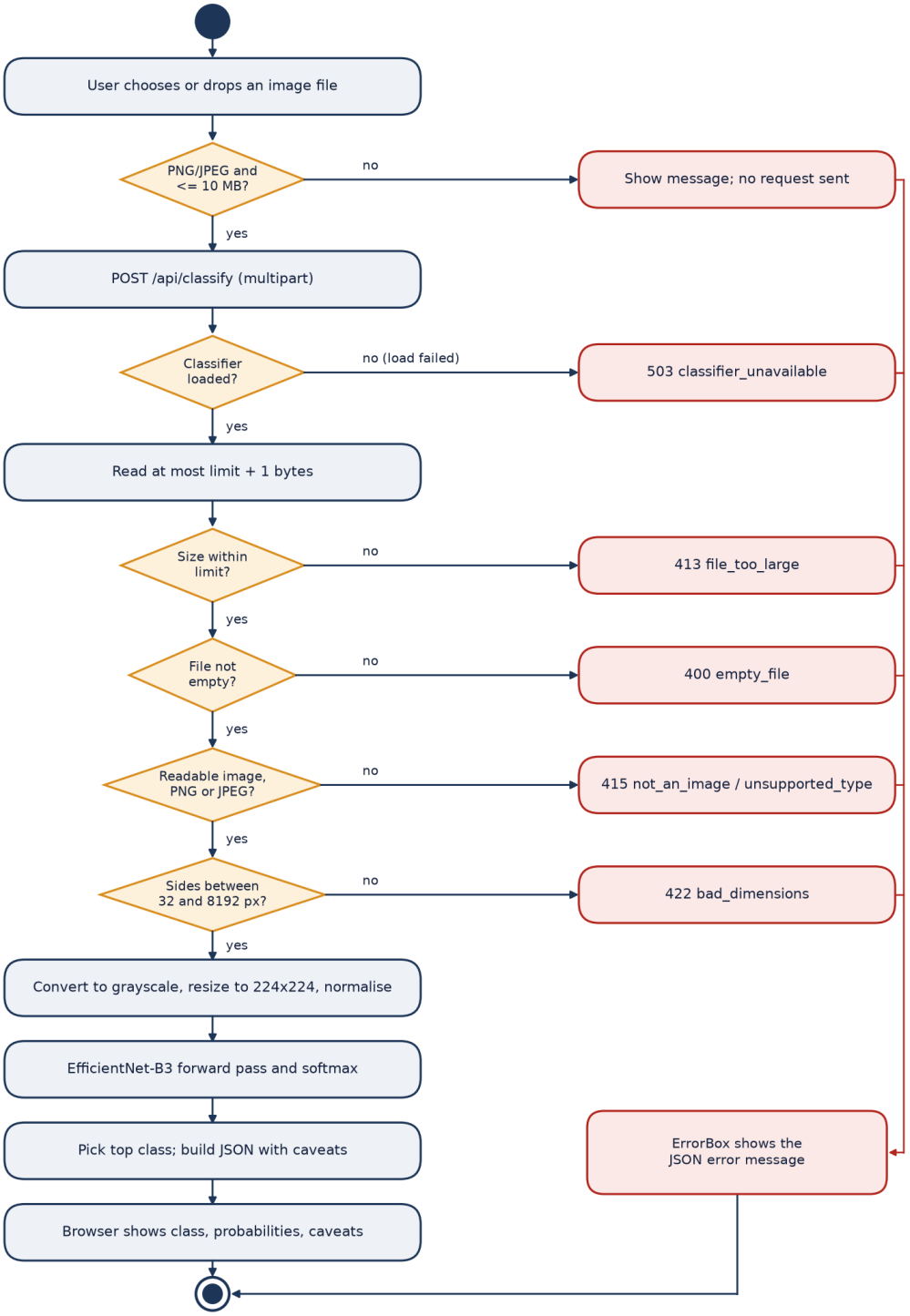


Figure B.5: Activity diagram of an image upload: the checks that decide between an error message and a classification result.

The diagram shows the order of the checks. The model-availability check comes first so that a request is refused before the file is read; the size check reads at most one byte more than the limit, so an oversized file is never held in memory in full.

B.3 Algorithms

Five algorithms are central to the project. Algorithms B.1 and B.2 belong to the research pipeline (Chapter 3); B.3 and B.4 are executed by the application; B.5 defines the evaluation metrics. Symbols: τ is the similarity threshold, *n_critic* the number of critic updates per generator update and λ the gradient-penalty weight.

Algorithm B.1: Near-duplicate clustering and cluster-disjoint split

Input: pooled images P (7,200), threshold $\tau = 0.98$, ratios 70/15/15, seed 42

Output: partitions Train, Validation and Test with no cluster in more than one

for each image i: $v_i \leftarrow$ 32×32 grayscale, mean removed, divided by its length; $h_i \leftarrow$ SHA-256 of the file
for each pair (i, j):
    $s_{ij} \leftarrow$ maximum over the 8 orientations o of the dot product $v_i \cdot o(v_j)$ (identity, flips, rotations, transposes)
    if $s_{ij} \geq \tau$ **or** $h_i = h_j$: union(i, j)
clusters ← connected components of the union-find structure
quarantine clusters whose images carry conflicting class labels, and clusters made only of files marked *aug*
rep$(c) \leftarrow$ one deterministic representative per remaining cluster (non-augmented, originally Training preferred)
for each class: assign its representatives, cluster by cluster, to Train / Validation / Test in the ratio 70/15/15
check that no cluster occurs in more than one partition; write the manifest with file hashes

Cost: the comparison of all pairs in 8 orientations is $O(8 \cdot n^2 \cdot d)$ with d = 1,024 pixels, computed as matrix products. The method finds copies, mirrors and re-encodings; it cannot find different slices of the same patient (Section 3.3).

Algorithm B.2: Training of the conditional WGAN-GP with a projection critic

Input: training partition (2,311 images with labels), *n_critic* = 5, $\lambda = 10$, Adam (learning rate 1×10^{-4}, $\beta = 0.5, 0.9$), batch 16, 3,000 generator steps

Output: generator $G(z, y)$ saved as a checkpoint

for $g = 1 \ldots 3{,}000$:
    repeat *n_critic* times:
        draw a real batch (x, y) and noise $z \sim N(0, I)$; $\tilde{x} \leftarrow G(z, y)$
        draw $\varepsilon \sim U(0, 1)$; $\hat{x} \leftarrow \varepsilon\, x + (1 - \varepsilon)\, \tilde{x}$ (the label y is held fixed)
        $GP \leftarrow$ mean of $(\|\text{gradient of } D(\hat{x}, y) \text{ with respect to } \hat{x}\| - 1)^2$
        $L_D \leftarrow$ mean $D(\tilde{x}, y)$ − mean $D(x, y) + \lambda \cdot GP$; update D (the critic)
    draw noise z and labels y; $L_G \leftarrow -$ mean $D(G(z, y), y)$; update G
    every 500 generator steps: save the checkpoint (weights, optimiser states, counters, class mapping)
projection critic: $D(x, y) = \psi(h(x)) + E(y) \cdot h(x)$, with no batch normalisation

Cost: 15,000 critic updates for 3,000 generator updates. The first run stopped at step 2,861 with a non-finite loss and was resumed from step 2,500 (Section 3.5).

Algorithm B.3: Seeded class-conditional generation (application)

Input: class c, count n (1 to 16), optional seed s

Output: n PNG images, the seed used

1. **if** c is not one of the four classes **or** n is outside 1 … 16 **or** s is outside 0 … $2^{31}-1$: return error 422
2. **if** the generator is not loaded and cannot be loaded: return error 503 with the reason
3. **if** s is absent: $s \leftarrow$ a fresh random integer
4. $z \leftarrow n$ vectors of 100 numbers from N(0, I), drawn on the CPU with a generator seeded by s; $y \leftarrow$ the index of c repeated n times
5. $x \leftarrow G(z, y)$ in evaluation mode, without gradients; values in [−1, 1]
6. pixel $\leftarrow$ round($(x + 1) \times 127.5$), limited to 0 … 255; encode each image as an 8-bit grayscale PNG, then as a base64 data URI
7. return the images, s, the model name, the checkpoint, *synthetic* = true and the caveats

Cost: one forward pass of n images; 24.1 ms for 8 images on the Apple GPU in Figure 4.3.

Algorithm B.4: Validation and classification of an uploaded image (application)

Input: uploaded file

Output: predicted class with the probability of the four classes, or a JSON error

1. **if** the classifier is not loaded and cannot be loaded: error 503
2. *data* $\leftarrow$ read at most (limit + 1) bytes; **if** longer than the limit: error 413; **if** empty: error 400
3. open *data* as an image; **if** it cannot be read: error 415 (not an image); **if** the format is not PNG or JPEG: error 415
4. **if** the shorter side is below 32 or the longer side above 8,192 pixels: error 422
5. $g \leftarrow$ grayscale of the image, resized to 224×224 (bilinear); scale to [0, 1]; $t \leftarrow (g - 0.1856) / 0.1892$
6. $p \leftarrow$ softmax(*EfficientNet-B3*(t)) in evaluation mode, without gradients
7. return the class with the largest p, all four p, the time, the device and the caveats

Cost: one forward pass; 143.3 ms in Figure 4.4. There is no out-of-distribution test: any readable image receives one of the four labels.

Algorithm B.5: Evaluation metrics and the matched-size control

Input: real test images, generated images, training sets of conditions A, B, C

Output: FID, KID and the three training sets

1. **FID**: from the 2,048-dimensional Inception features, μ_r, Σ_r (real) and μ_g, Σ_g (generated); FID ← $\|\mu_r - \mu_g\|^2$ + trace($\Sigma_r + \Sigma_g - 2\,(\Sigma_r \Sigma_g)^{1/2}$)
2. **KID**: for 100 random subsets of 300 real and 300 generated images, the unbiased squared maximum mean discrepancy with the kernel $k(a, b) = (a \cdot b / 2{,}048 + 1)^3$; report the mean × 1,000
3. repeat the generation for seeds 1001, 1002, 1003; report the mean and standard deviation; add the real-versus-real reference floor
4. **Condition A** ← the 2,311 real training images
5. **Condition B** ← A plus 722 cWGAN-GP images per class (target counts 1,640; 1,288; 949; 1,322 = 5,199 images)
6. **Condition C** ← for each class, draw real training images **with replacement** until B's count for that class is reached
7. **for** condition in A, B, C and seed in 42, 43, 44: train the CNN; pick the epoch on validation accuracy; read the test partition once

Condition C separates the effect of synthetic images from the effect of simply having more training images (Section 3.7).

B.4 User Interface Design

B.4.1 Design principles and visual identity

The interface is a calm, data-first dashboard. Navy is used for structure, teal for emphasis and result panels, and amber for every research-prototype notice, so that a warning always has the same colour. The values in Table B.5 are the theme variables of the frontend; cards, buttons, badges and alerts come from the shadcn/ui component set and the charts from Recharts.

Table B.5: Design tokens of the interface.

Token	Value	Use
Primary (navy)	#1F3A5F	Header, primary buttons, headings, chart bars
Accent (teal)	#2A9D8F	Selected states, result panels, probability bars, chart bars
Background / card	#F6F8FA / #FFFFFF	Page background / cards
Warning (amber)	#FDF1DC, border #C98A1A	Research-prototype notice, SYNTHETIC badge
Destructive (red)	#B3261E	Error messages
Border / muted text	#D5DDE6 / #556070	Card outlines / secondary text
Shape and type	radius 0.625 rem; Inter, else the system sans-serif	Cards, buttons / all text

B.4.2 Layout and navigation

All pages share one layout: a sticky navy header with the product name, a “research prototype” tag and the main navigation, the amber notice, the page content and a footer. The five routes and the API calls behind them are shown in Figure B.6. An unknown address shows a “Page not found” message inside the same layout.

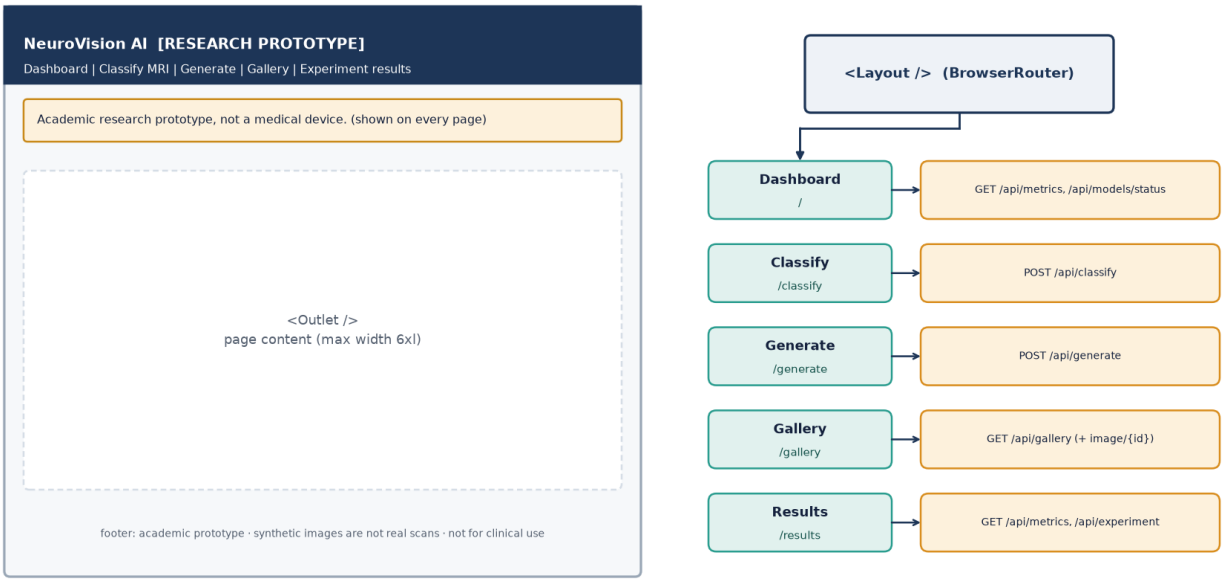


Figure B.6: Common page layout (left) and route map with the API calls of each page (right).

B.4.3 Page designs

Table B.6 lists, for each page, its main components, its data source and the states it handles. Figures B.7 and B.8 are wireframes of the five pages; Figures 4.2 to 4.5 in Chapter 4 are screenshots of the finished pages.

Table B.6: Page designs: components, data and states.

Page	Main components	Data source	States handled
Dashboard	Four key-figure tiles; FID chart; A/B/C chart; model-readiness card; links to the other pages	GET /api/metrics, /api/models/status	loading, error, model unavailable
Classify MRI	Drop zone and file picker with preview; Classify and Clear buttons; result panel with class and four probability bars; caveats	POST /api/classify	idle, file rejected, busy, result, server error, classifier unavailable
Generate	Class buttons (toggle); image count; seed field with inline validation; image grid with SYNTHETIC badge; per-image and grid download; caveats	POST /api/generate	idle, invalid seed, busy, result, error, generator unavailable
Gallery	Cards with the saved figure sheet, a real / synthetic / mixed badge and a provenance note	GET /api/gallery (+ image/{id})	loading, error, empty

Page	Main components	Data source	States handled
Experiment results	FID and KID charts and table; A/B/C chart and table; class-wise F1; dataset, methodology and limitations	GET /api/metrics, /api/experiment	loading, error (result file missing)

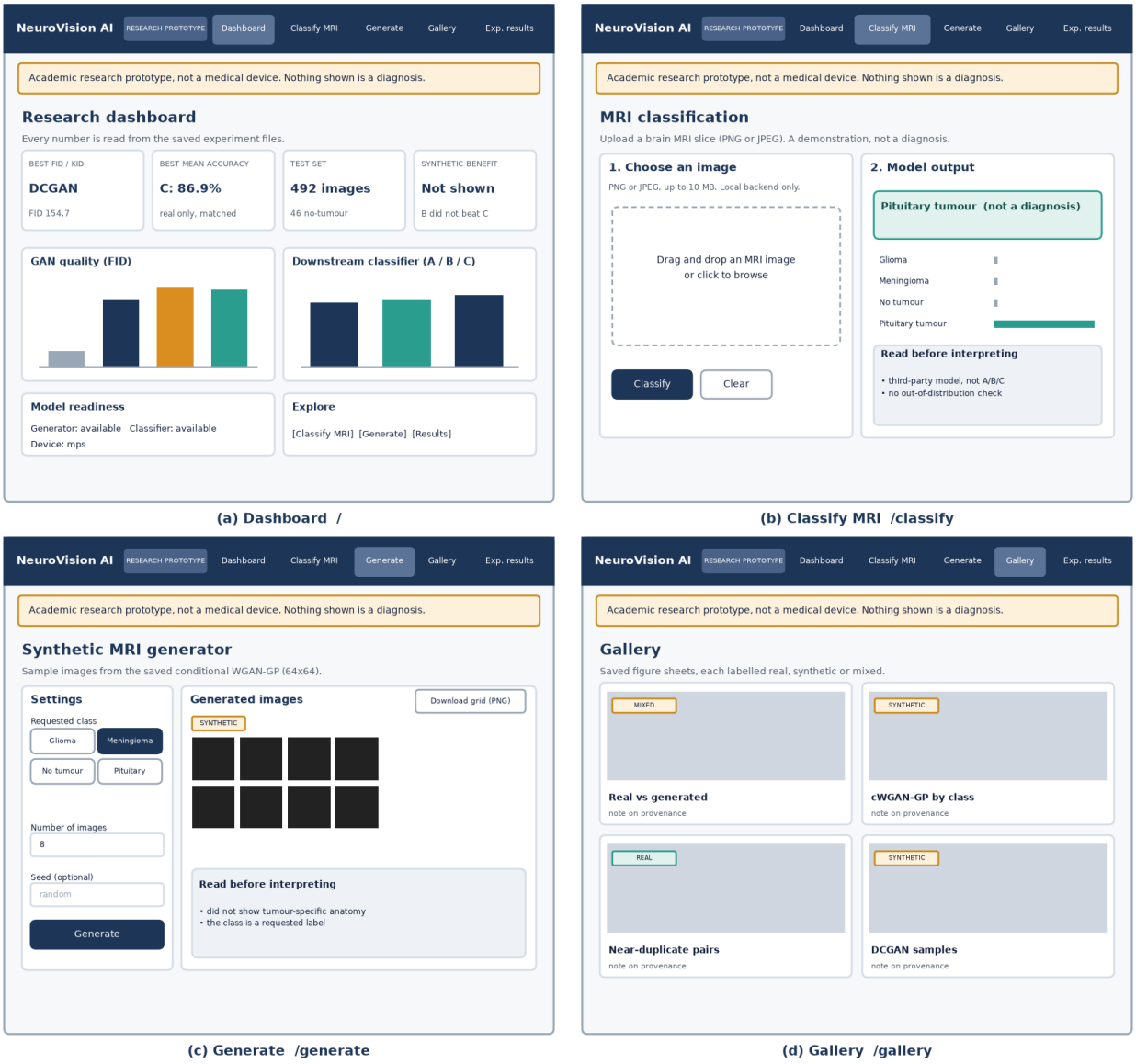


Figure B.7: Wireframes of the Dashboard, Classify MRI, Generate and Gallery pages.

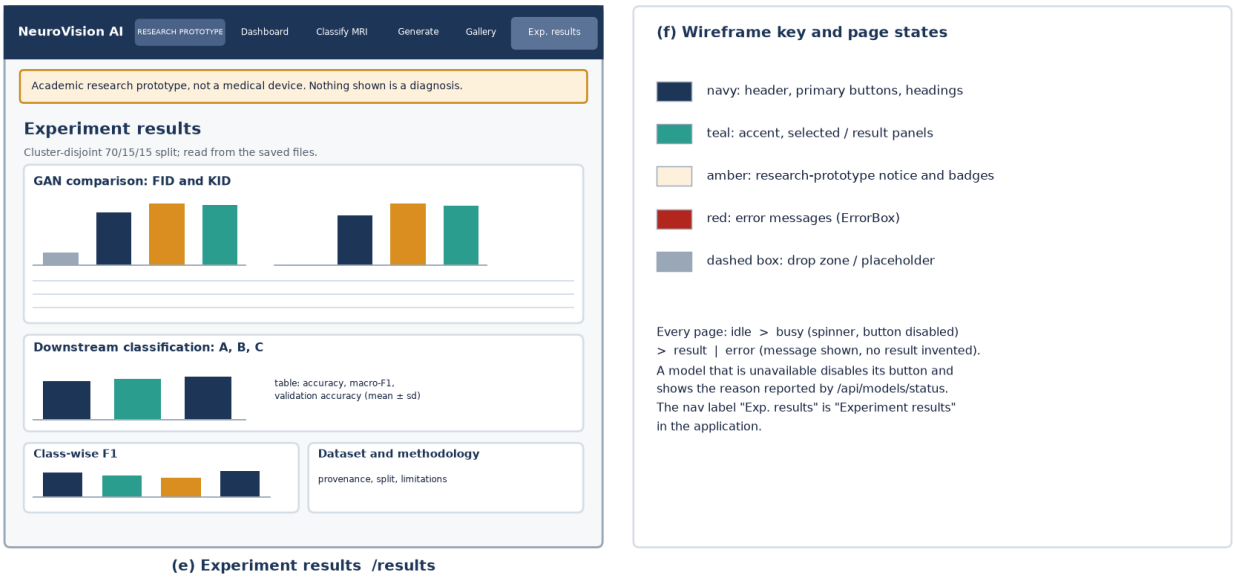


Figure B.8: Wireframe of the Experiment results page and the key to the wireframes.

B.4.4 Feedback, accessibility and responsiveness

- **Feedback.** A spinner and a disabled button show that a model is running; failures appear in an error box with the server's message; a model that is unavailable disables its button and shows the reason.
- **Accessibility.** The navigation and the list of generated images carry labels; the drop zone is a focusable button that opens with Enter or Space; class buttons expose their pressed state; the seed field marks itself invalid; the generated-image area is announced politely; decorative icons are hidden from screen readers.
- **Responsiveness.** The navigation wraps, the two-column pages (Classify, Generate) stack on narrow screens, and the Gallery and the generated-image grid change their number of columns with the width.
- **Safety messaging.** The amber notice is on every page, generated images carry a SYNTHETIC badge and the classification result is titled “not a diagnosis”.

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

EVALUATION REPORT

S. No	Evaluation Parameter	Max. Marks	Marks Secured
1	Certification Course (AI / ML in Healthcare – NPTEL, Coursera, etc.)	10	
2	Healthcare AI Case Study	20	
2a	Understand the clinical problem and its context	4	
2b	Identify the healthcare data (type, source, preprocessing, privacy)	4	
2c	Select and justify the AI/ML/DL technique	4	
2d	Evaluate the results and their clinical relevance (accuracy, sensitivity, specificity)	4	
2e	Address ethical, regulatory and patient-safety concerns (bias, consent, data protection)	4	
3	PPT Presentation / Demonstration of the Case Study	10	

Signature of Faculty In-charge
Ms. Gethsia P
Assistant Professor
Division of Artificial Intelligence and Machine Learning